

Teórico Programación 4

FING – 2024

Contenido

1. INTRODUCCIÓN	4
1.1. ANÁLISIS	4
1.1.1. <i>Modelo de Dominio</i>	4
1.1.2. <i>Diagramas de Secuencia</i>	5
1.1.2.1. Contrato	5
1.2. DISEÑO	6
1.2.1. <i>Diagramas de Comunicación</i>	6
1.2.2. <i>Diagrama de Clases</i>	6
1.3. IMPLEMENTACIÓN	7
1.4. RESUMEN	7
2. ANÁLISIS	8
2.1 CASOS DE USO	9
2.2 MODELO DE DOMINIO	10
2.2.1. <i>Conceptos</i>	10
2.2.1.1. Atributos	10
2.2.1.2. Identificación de Conceptos	12
2.2.2. <i>Asociaciones</i>	13
2.2.2.1. Multiplicidades	13
2.2.2.2. Roles	14
2.2.2.3. Agregación	15
2.2.2.4. Generalizaciones	15
2.2.2.5. Tipos Asociativos	16
2.2.3. <i>Restricciones</i>	17
2.3. DIAGRAMAS DE SECUENCIA DEL SISTEMA	18
2.3.1. <i>Contratos</i>	20
3. DISEÑO	21
3.1. COLABORACIÓN	21
3.2. DIAGRAMAS DE COMUNICACIÓN	22
3.2.1. <i>Clases</i>	22
3.2.2. <i>Instancias</i>	22
3.2.3. <i>Links</i>	22
3.2.4. <i>Mensajes</i>	23
3.2.4.1. Sintaxis de los Mensajes	23
3.2.5. <i>Iteración</i>	23
3.2.6. <i>Creación de Instancias</i>	23
3.2.7. <i>Destrucción de Instancias</i>	24
3.2.8. <i>Números de Secuencia</i>	24
3.2.9. <i>Mensajes Condicionales</i>	24
3.2.10. <i>Colecciones</i>	25
3.2.10.1. Responsabilidad de las Colecciones	25
3.2.11. <i>Datatypes</i>	26
3.2.12. <i>Visibilidad</i>	27
3.2.12.1. Association	27
3.2.12.2. Parameter	27
3.2.12.3. Local	28
3.2.12.4. Global	28
3.2.13. <i>Criterios GRASP</i>	29
3.2.13.1. Expert	30
3.2.13.2. Creator	30
3.2.13.3. Bajo Acoplamiento	30
3.2.13.4. Alta Cohesión	31
3.2.13.5. No Hables con Extraños	31

3.2.13.6. Controller	32
3.3. PROGRAMACIÓN EN CAPAS	35
3.3.1. Interfaces del Sistema	35
3.3.2. Fabricas	37
3.4. DIAGRAMA DE CLASES DE DISEÑO (DCD)	38
3.4.1. Inclusión de Operaciones	41
3.4.1.1. Operación de Creación.....	41
3.4.1.2. Operaciones de Acceso.....	41
3.4.2. Inclusión de Colecciones.....	41
3.4.3. Tipos Asociativos	41
3.5. PATRONES DE DISEÑO	42
3.5.1. Factory.....	42
3.5.2. Singleton	43
3.5.3. State	43
3.5.4. Strategy.....	44
3.5.5. Composite.....	45
3.5.6. Template Method	46
3.5.7. Proxy.....	47
3.5.8. Observer	48
3.5.9. Adapter	49
4. IMPLEMENTACIÓN	50
4.1. INTERFACES.....	51
4.1.1. Realización	52
4.2. CLASES.....	53
4.3. RELACIONES	54
4.3.1. Generalizaciones	54
4.3.2. Realizaciones	54
4.3.3. Asociaciones	54
4.4. INTERACCIONES	55
4.5. MANEJO DE OBJETOS	56
4.5.1. Referencias	56
4.5.2. Copias	57
4.5.3. Destrucción.....	58
4.6. COLECCIONES.....	58
4.6.1. Colecciones Genéricas.....	58
4.6.2. Iteradores	59
4.6.3. Diccionarios	59
4.6.4. Búsquedas	59
4.6.5. Contenedores STL	59
4.6.5.1. Set.....	60
4.6.5.2. Map.....	61
4.7. PATRONES DE DISEÑO – IMPLEMENTACIÓN	62
4.7.1. Singleton	62
4.7.2. Factory.....	63
4.7.3. State	66
4.7.4. Strategy.....	68
4.7.5. Composite.....	69
4.7.6. Template Method	71
4.7.7. Proxy.....	73
4.7.8. Observer	74
4.7.9. Adapter	75

1. Introducción

La Programación Orientada a Objetos (POO) es un paradigma de programación que utiliza "objetos" – entidades que combinan datos y comportamientos – como unidades fundamentales para construir aplicaciones de software. Puede ser entendida como una forma de organizar el software como una colaboración de objetos que interactúan entre si por medio de mensajes, es decir, es un enfoque basado en abstracciones de conceptos existentes del mundo real.

Dividimos el proceso de desarrollo orientado a objetos en tres etapas:

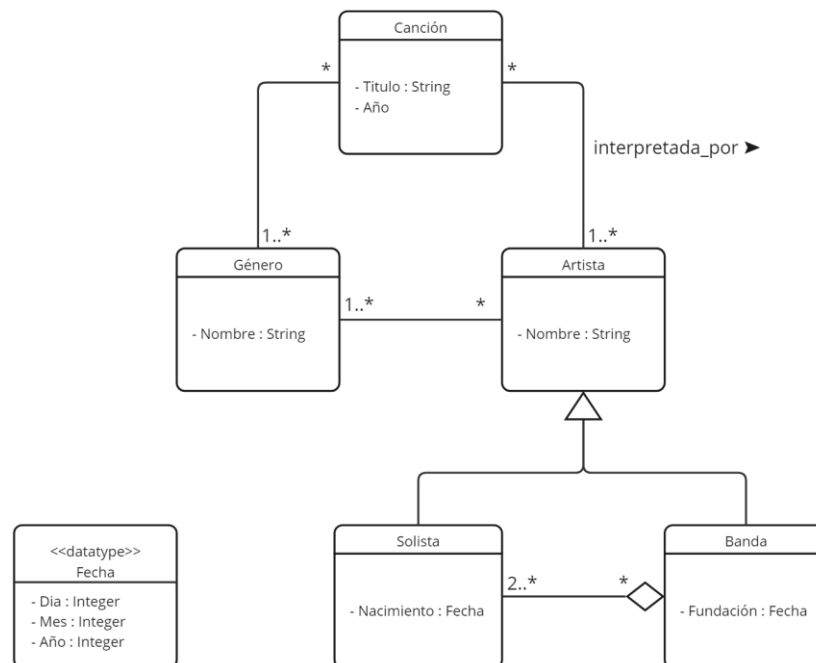
- Análisis (Orientado a objetos)
- Diseño (Orientado a objetos)
- Implementación (Orientada a objetos)

1.1. Análisis

Conlleva un proceso de identificación y abstracción de conceptos del dominio del problema. Utilizamos el Modelo de Dominio y los Diagramas de Secuencia del Sistema como una primera aproximación al problema.

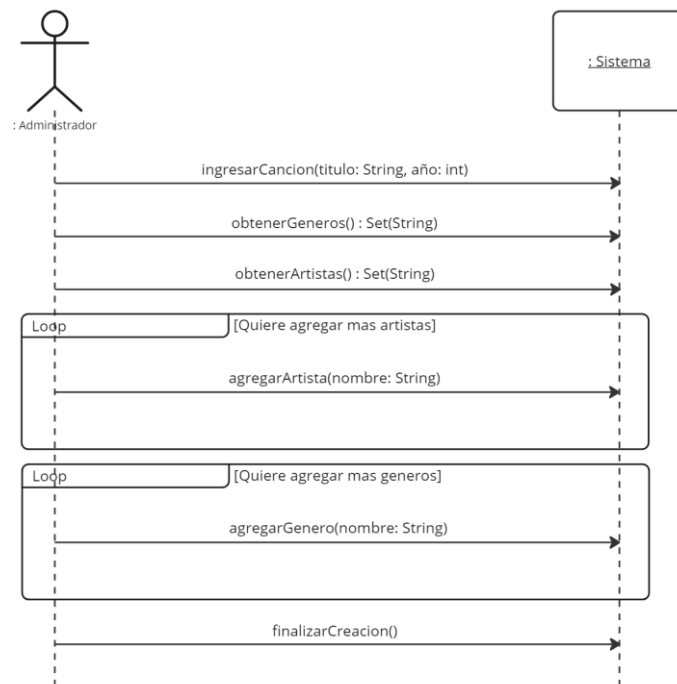
1.1.1. Modelo de Dominio

Contiene los conceptos y sus relaciones. La información para realizarlos es provista, principalmente, por los Casos de Uso. Se incluyen además restricciones a las cuales está sujeto el dominio.



1.1.2. Diagramas de Secuencia

Los Diagramas de Secuencia (DSS) ilustran la forma en que los actores realizan “invocaciones” sobre el sistema. Los diferentes escenarios de uso son los definidos por los Casos de Uso. Asimismo, el efecto de cada mensaje es especificado en forma precisa por medio de un contrato.



1.1.2.1. Contrato

Especifica en forma concreta y precisa el efecto de cada mensaje sin entrar en detalles de como se realiza cada mensaje.

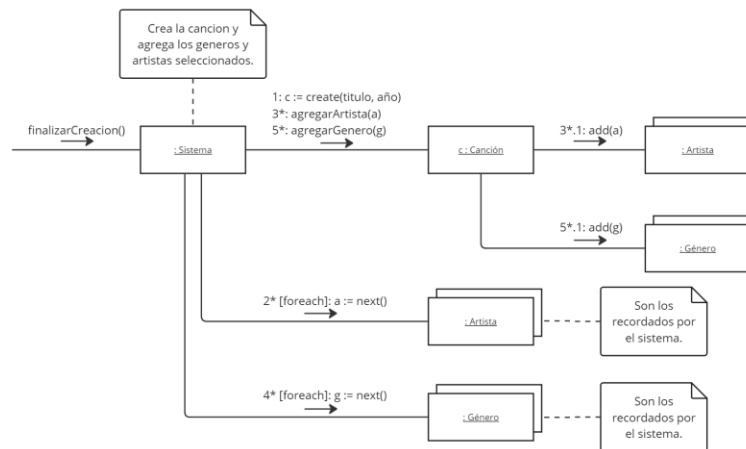
Operación	ingresarCancion(título: String, año: int)
Descripción	Comienza el ingreso de una nueva canción al sistema especificando su información básica.
Precondiciones	1. No existe una canción de nombre igual a título. 2. El parámetro año no es mayor al año actual.
Postcondiciones	El sistema recuerda el titulo y el año especificados, para usarse posteriormente.

1.2. Diseño

En base a los conceptos candidatos encontrados durante la etapa de análisis y por medio de ciertos principios y técnicas se definen los objetos lógicos y la forma de comunicación entre ellos para una posterior implementación (programación), para ello, recurrimos a los Diagramas de Comunicación y Diagrama de Clases del Sistema.

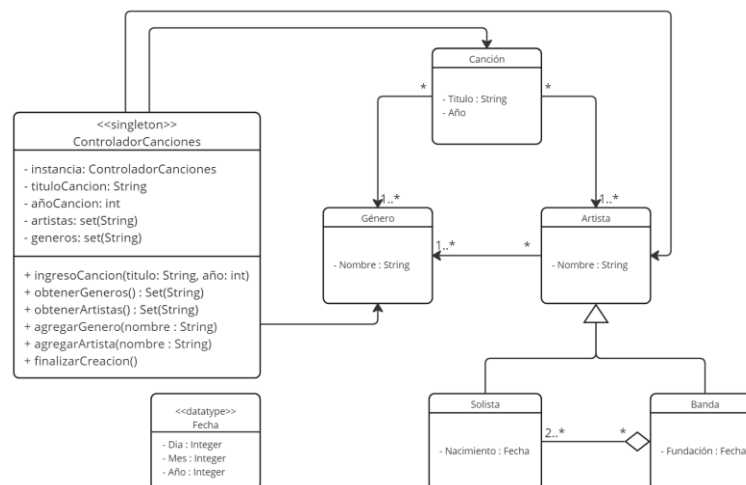
1.2.1. Diagramas de Comunicación

Se realiza un Diagrama de Comunicación por operación del sistema (ver: [DSS](#)) con el fin de definir como se comunican los objetos para resolver dichas operaciones.



1.2.2. Diagrama de Clases

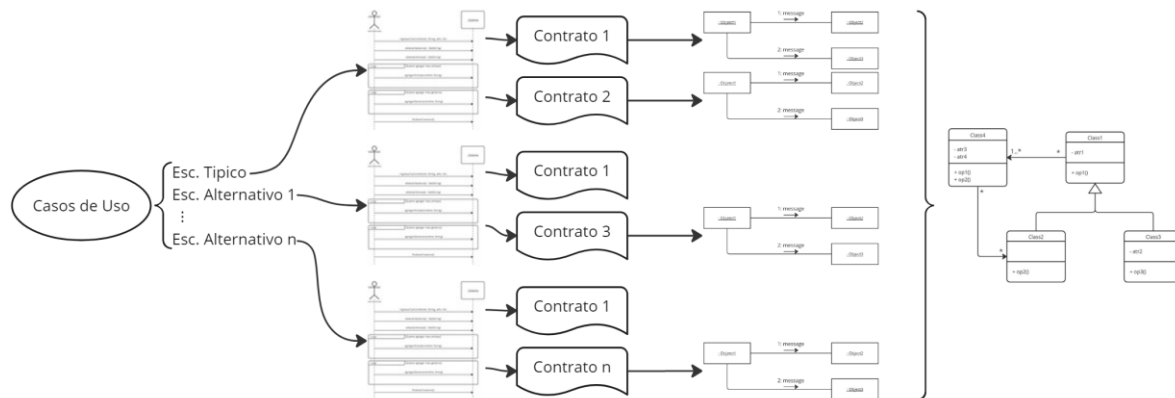
Especifica la estructura necesaria para que todas las interacciones puedan ocurrir, resumiendo la información provista por los Diagramas de Comunicación mientras que su estructura general está guiada por la del Modelo de Dominio (ver: [Modelo de Dominio](#))



1.3. Implementación

Su objetivo es codificar en un lenguaje de programación orientado a objetos las construcciones definidas en las etapas previas.

1.4. Resumen



2. Análisis

La esencia del análisis orientado a objetos es la descomposición del problema en conceptos individuales. Un Modelo de Dominio contiene principalmente los conceptos y sus relaciones que sean significativos en el dominio del problema.

En el desarrollo de software, uno de los elementos más críticos es la especificación de requerimientos. Esta especificación actúa como un contrato entre los usuarios y los desarrolladores, definiendo qué es lo que debe hacer el sistema para satisfacer las necesidades de los usuarios. La correcta identificación y documentación de estos requerimientos son vitales para el éxito del proyecto, ya que de ellos depende el diseño, la implementación y las pruebas del sistema.

Los requerimientos de software se dividen en dos grandes categorías:

- **Requerimientos Funcionales:** Describen las funciones específicas que el sistema debe realizar. Estos incluyen interacciones entre el sistema y sus usuarios, así como el comportamiento esperado del sistema en distintas situaciones.
- **Requerimientos No Funcionales:** Se refieren a atributos de calidad que el sistema debe poseer, como la usabilidad, confiabilidad, performance, y restricciones técnicas o de implementación.

Cuando los sistemas y sus interacciones comenzaron a volverse más complejos, la simple enunciación de funciones del sistema dejó de ser suficiente. En respuesta a esta complejidad, surgió el enfoque de casos de uso.

2.1 Casos de Uso

Un caso de uso es una descripción detallada de cómo un usuario o un grupo de usuarios (denominados actores) interactúan con un sistema para lograr un objetivo específico. Este enfoque se basa en la noción de actor, que puede ser un humano, otro sistema o cualquier entidad externa que interactúa con el sistema.

Componentes de un Caso de Uso:

- **Nombre del Caso de Uso:** Un identificador único y descriptivo.
- **Actores:** Los participantes externos que interactúan con el sistema.
- **Sinopsis:** Una breve descripción del objetivo del caso de uso.
- **Curso Típico de Eventos:** La secuencia de pasos más común que siguen los actores para alcanzar su objetivo.
- **Cursos Alternativos:** Variantes y excepciones posibles durante la interacción.

Nombre	Escribir Email
Actor	Usuario
Descripción	El caso de uso comienza cuando el Usuario desea escribir un correo electrónico. Para ello especifica el cuerpo del correo electrónico y un asunto. Luego si lo desea el usuario puede adjuntar uno o más archivos de su disco duro en el email. Al finalizar de escribir el email el sistema especifica una lista de contactos y el usuario selecciona opcionalmente uno o más destinatarios. Para cada destinatario puede especificar que tipo de destinatario se trata (Principal, Con Copia o Con Copia oculta). Luego el usuario puede elegir más destinatarios escribiendo manualmente sus correos electrónicos y el tipo de destinatario al igual que lo hizo antes. Al finalizar el sistema envía el correo y lo copia en la carpeta 'Enviados'.

Beneficios de los Casos de Uso:

- **Claridad:** Facilitan la comprensión de los requerimientos por parte de todos los interesados, incluidos aquellos sin conocimientos técnicos.
- **Enfoque en Objetivos:** Ayudan a centrar el desarrollo en las necesidades y objetivos de los usuarios.
- **Iterativos e Incrementales:** Permiten la identificación y especificación de nuevos casos de uso a medida que el proyecto avanza.

Un caso de uso bien definido proporciona una visión completa y clara de cómo se utilizará el sistema, asegurando que todas las interacciones necesarias sean consideradas y documentadas.

2.2 Modelo de Dominio

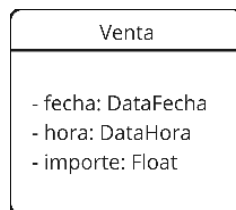
Nos concentramos en encontrar y describir los conceptos en el dominio de la aplicación, para ello, se busca entender cada caso de uso en términos de intercambios de mensajes entre los actores y el sistema.

Un Modelo de Dominio contiene, principalmente, los conceptos y sus relaciones que sean significativos en el dominio del problema, donde el problema y los requerimientos determinan qué es significativo.

2.2.1. Conceptos

Un concepto se refiere a una entidad o idea clave que existe dentro del dominio del problema que se está modelando. Estos conceptos representan los elementos fundamentales que describen la realidad del negocio o del sistema que se desea implementar.

Ejemplo:

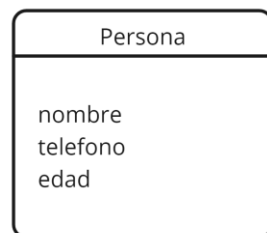


En un sistema de gestión de ventas, un cliente es un concepto que representa a una persona o empresa que compra productos. Mientras que una venta es un concepto que representa una solicitud de productos realizada por un cliente.

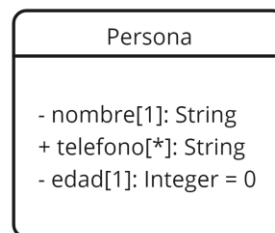
2.2.1.1. Atributos

Es necesario identificar aquellos atributos que permitan satisfacer los requerimientos de información. Al mostrar un atributo es necesario especificar, al menos, su nombre, luego, se incluyen otras propiedades opcionales tales como:

- Tipo
- Multiplicidad
- Valor Inicial
- Visibilidad



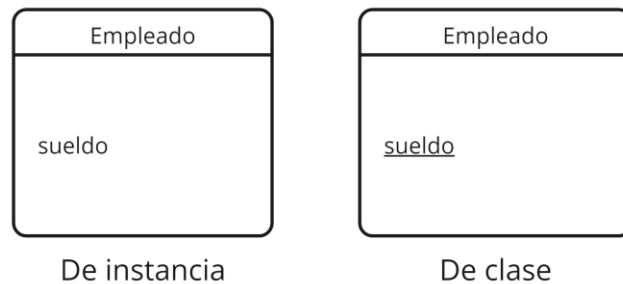
Representacion
minima



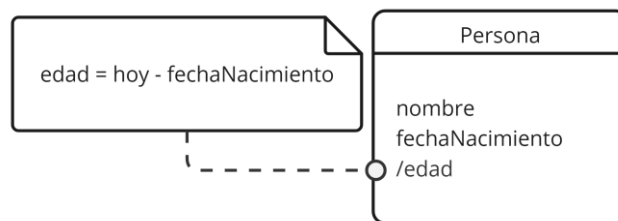
Representacion
completa

También se considera el alcance de un atributo dado:

- **De clase:** Todas las instancias de dicha clase comparten el mismo valor (no es una constante, solo es común para todas las instancias)
- **De instancia:** Cada instancia tiene su propia copia de ese atributo, permitiendo valores distintos para cada una.



Por otro lado, un atributo que sea derivable (calculable), se marca con “/” usualmente adjuntándosele una nota especificando la forma en que se le calcula.



Asimismo, podemos considerar las siguientes sugerencias:

- No utilizar atributos como clave foránea para relacionar elementos.



- De ser necesario es posible definir tipos no-primitivos (Ej.: DataFecha) para un problema. Los tratamos como tipos primitivos en el sentido de que cualquiera de las dos implementaciones anteriores (utilizarlo como atributo o mediante flecha) son correctas.

2.2.1.2. Identificación de Conceptos

Para comenzar a construir un Modelo de Dominio es necesario identificar en primer lugar los conceptos participantes, para ello, utilizamos alguno de los siguientes métodos:

- **Lista de Categorías de Conceptos:** Consiste en repasar la lista de categorías de conceptos buscando conceptos del dominio del problema que apliquen a cada categoría. Algunas de las categorías más importantes son:
 - Objetos físicos o tangibles
 - Descripciones de cosas
 - Lugares
 - Transacciones
 - Roles
- **Identificación de Sustantivos:** Se identifican los sustantivos de una descripción textual del problema (Casos de Uso) y se los considera como conceptos o atributos candidatos.

Luego de identificados los conceptos se procede a incluirlos en el Modelo de Dominio, se agregan las asociaciones necesarias para registrar las relaciones que necesiten ser preservadas, así como los atributos necesarios para satisfacer los requerimientos de información.

Obs.: Es mejor sobre-especificar que sub-especificar pues el costo de eliminar un concepto que resultó innecesario es menor que el de agregar uno que fue omitido.

Ejercicio:

Supongamos que una instancia de “Producto” representa a un producto físico en una tienda; un producto tiene un numero de serie, una descripción, un precio y un código, datos los cuales no aparecen en ninguna otra parte.

Problemas:

- En el caso de que un producto se agote nadie podría saber cual era el precio de ese producto dado que esa información estaba incluida únicamente en las instancias que conformaban el inventario.
- Información repetida.

Solución:

Se necesitan conceptos que sean descripciones de otros conceptos. En el caso de “Producto” necesitamos una “DescripcionProducto” que registre la información de los productos. Estos conceptos no representan un producto, sino información acerca de ellos.



2.2.2. Asociaciones

Es necesario identificar aquellas asociaciones entre conceptos que sean necesarias para satisfacer los requerimientos de información y que ayuden a la comprensión del Modelo de Dominio.

Como método de Detección de Asociaciones utilizamos la lista de categorías de asociaciones comunes. Algunas de las categorías más comunes son:

- A es una parte física de B
- A es una parte lógica de B
- A está contenido físicamente en B
- A está contenido lógicamente en B
- A es un miembro de B
- A es una descripción de B
- A es conocido/registrado/capturado en B
- A usa o maneja B
- A se comunica con B
- A es propiedad de B

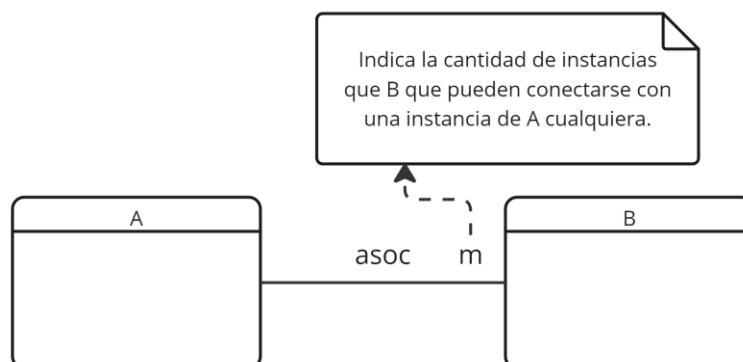
Dichas asociaciones las separamos en dos categorías:

- **De comprensión:** Permiten comprender mejor el problema.
- **Need-to-know:** Permiten satisfacer los requerimientos de información.

Idealmente a la hora de diseñar un Modelo de Dominio se mantiene un punto intermedio entre un modelo que contenga solo asociaciones need-to-know y otro con todas las asociaciones concebibles.

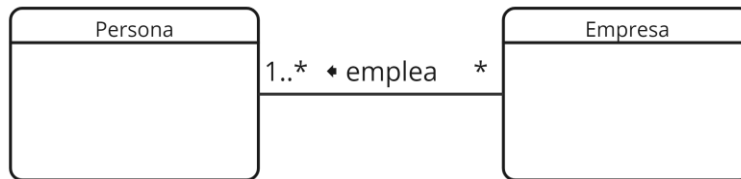
2.2.2.1. Multiplicidades

La multiplicidad limita la cantidad de veces que una instancia determinada está conectada a otras a través de una asociación. Ésta es indicada en el extremo de asociación opuesto.

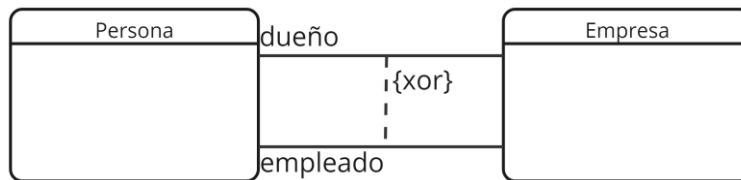


2.2.2.2. Roles

Especifican el papel que juegan las clases en una asociación dada. Pueden ser necesarios para eliminar ambigüedades.

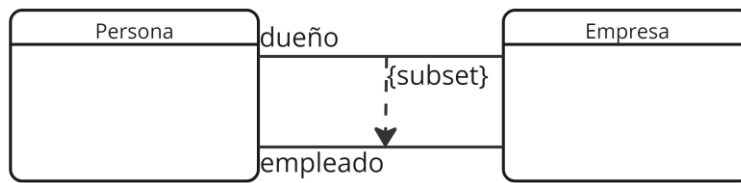


En ocasiones, si existen mas de un tipo de asociación entre dos conceptos, es necesario especificar que existe una restricción entre dos asociaciones.



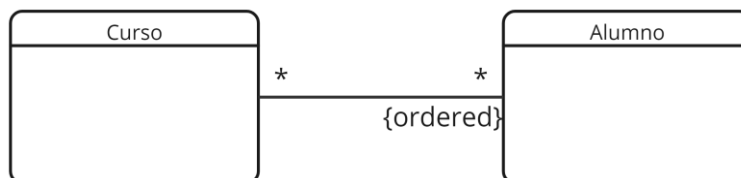
De esta forma una persona no puede ser dueño y empleado de la misma empresa.

Otro ejemplo, si dos instancias están conectadas por una asociación, también lo deben estar por otra asociación.



De esta forma una persona que sea la dueña de la empresa también tiene que ser empleado.

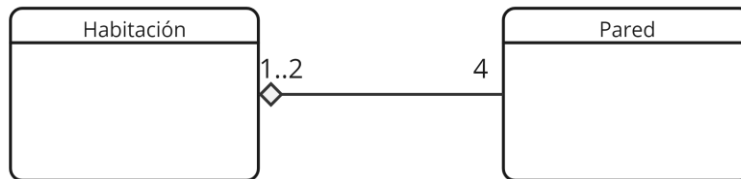
Por otro lado, también es posible indicar que existe un orden entre las instancias con las cuales otra instancia está relacionada.



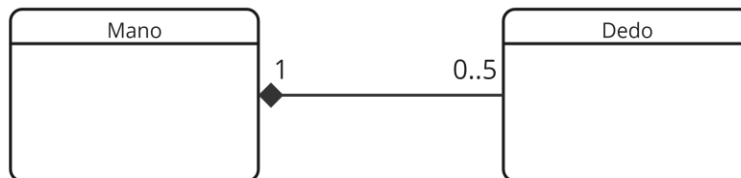
2.2.2.3. Agregación

Es una forma más fuerte de asociación, indica que un elemento es parte de otro. Existen dos tipos:

- **Agregación compartida:**



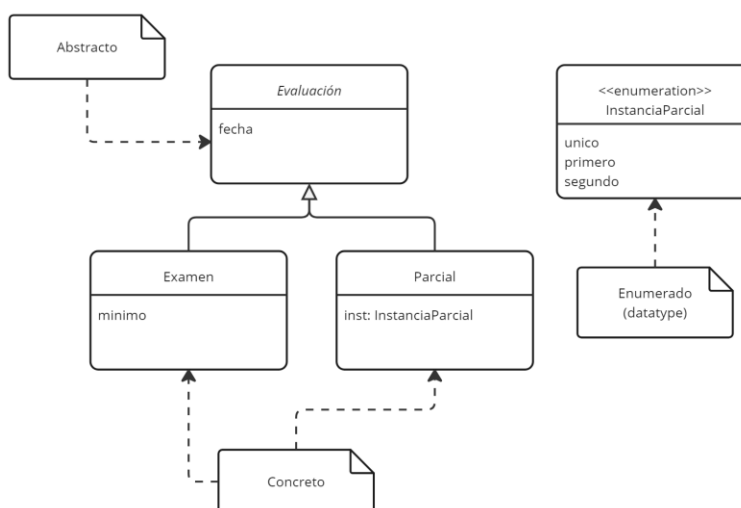
- **Agregación compuesta:** indica que un elemento es exclusivo del compuesto, haciendo que el máximo de la multiplicidad sea 1. Asimismo, en general, una acción sobre el compuesto se propaga a las partes, por ejemplo, la destrucción.



Agregacion compuesta

2.2.2.4. Generalizaciones

La generalización es una relación jerárquica entre conceptos en la que un concepto más general agrupa características comunes que son compartidas por uno o más conceptos más específicos.



2.2.2.5. Tipos Asociativos

Un tipo asociativo es un elemento que es tanto concepto como asociación. Un tipo asociativo es una entidad que se utiliza para representar una relación entre otros conceptos en el modelo de dominio, y puede incluir atributos que describen detalles específicos de esa relación.

Ejercicio:

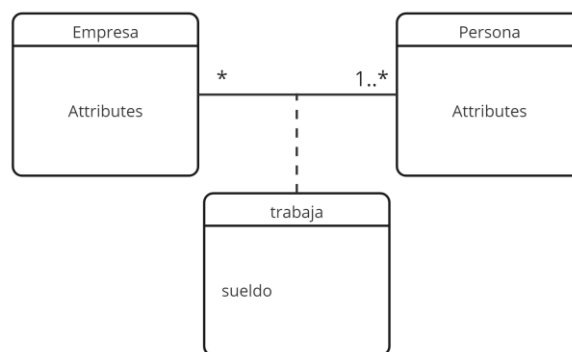
Una empresa contrata a diferentes personas para trabajar y a cada una le asigna un sueldo particular. Asimismo, una persona puede ser contratada por diferentes empresas, recibiendo sueldos diferentes por cada trabajo.

Problema:

- Incluir el sueldo en la persona no es correcto ya que una persona puede tener mas de un sueldo y este depende del trabajo.
- Incluir el sueldo en la empresa tampoco es correcto ya que la empresa paga sueldos distintos a cada empleado.

Solución:

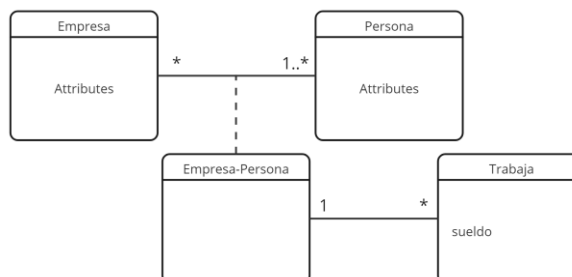
Implementar un tipo asociativo que para cada relación permite almacenar un valor distinto.



Problema Adicional:

- Supongamos a su vez que una persona puede tener varios sueldos dentro de una misma empresa por desarrollar distintas tareas.

Solución:



2.2.3. Restricciones

Es muy común el hecho de que un Modelo de Dominio no alcance a representar exactamente la realidad planteada. Para solucionar esto es posible realizar modificaciones al modelo, sin embargo, es posible que esto conlleve una reestructuración no menor del modelo, por lo que resulta más conveniente la imposición de restricciones (en particular invariantes).

Una invariante es un predicado que expresa una condición sobre los elementos del Modelo de Dominio y que siempre debe ser verdadero. Estas pueden ser expresadas informalmente en lenguaje natural.

Algunas de las restricciones más habituales son:

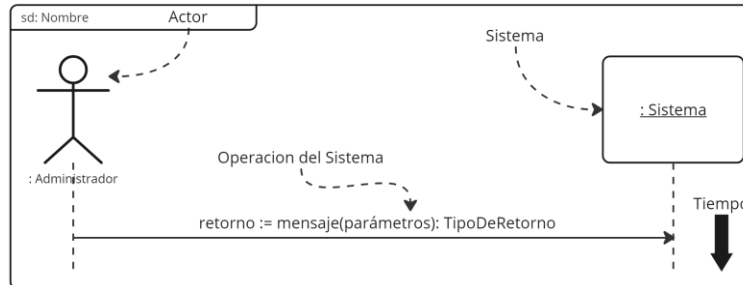
- **Unicidad de Atributos:** Un atributo tiene un valor único dentro del universo de instancias de un mismo tipo, permitiendo que una instancia sea identificada por ese valor.
- **Dominio de Atributos:** El valor de un atributo pertenece a cierto dominio.
- **Integridad Circular:** No puede existir circularidad en la navegación.
- **Atributos Calculados:** El valor de un atributo es calculado a partir de la información contenida en el dominio.
- **Reglas de Negocio:** Invariante que restringe el dominio del problema.

2.3. Diagramas de Secuencia del Sistema

El objetivo de los DSS es detallar la secuencia de eventos y mensajes intercambiados entre los actores y el sistema en un contexto específico, reflejando así el flujo de información y control que ocurre durante la ejecución de un caso de uso. En estos se incluye:

- Una instancia representando a cada participante (Sistema y Actores).
- Los mensajes enviados entre ellos en el/los escenario/s correspondiente/s.

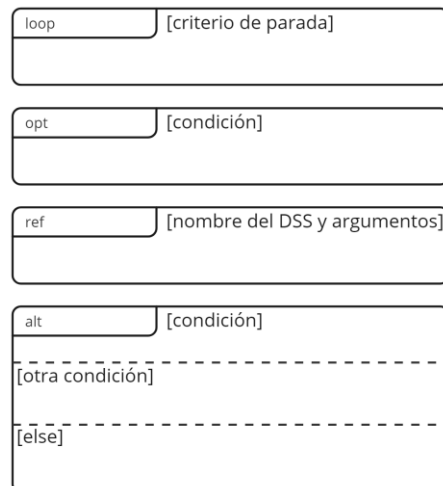
A continuación, la estructura general de un Diagrama de Secuencia:



Consideremos las siguientes sugerencias:

- El tipo de un parámetro o retorno puede ser tanto abstracto como un datatype definido a conveniencia.
- Utilizamos set(TipoDeDato) como parámetro o retorno cuando queremos referirnos a una colección de "TipoDeDato" y no una única instancia.

Por otro lado, contamos con varias estructuras de control que nos permiten representar dentro del diagrama de secuencia, bucles, condiciones, paralelismos, etc.:



Estas estructuras pueden encerrar uno o varios mensajes.

Por último, en los Diagramas de Secuencia del Sistema (DSS), las notas son elementos útiles para añadir comentarios, aclaraciones o detalles adicionales que no se pueden expresar fácilmente mediante los elementos estándar del diagrama. Las notas proporcionan contexto y explicaciones que ayudan a los lectores a entender mejor el diagrama y las interacciones representadas.

Propósito de las Notas:

- **Clarificación:** Añadir aclaraciones sobre ciertas interacciones o decisiones tomadas en el diagrama.
- **Contexto Adicional:** Proveer información contextual que puede no ser obvia a partir de los mensajes y actores presentes.
- **Descripciones Detalladas:** Incluir detalles adicionales sobre eventos específicos, condiciones, o secuencias de mensajes.
- **Referencias:** Apuntar a documentación externa, requisitos específicos, o detalles técnicos que apoyen la comprensión del diagrama.

Asimismo, cuando se trabaja con un sistema “con memoria”, se recae en la utilización de notas para especificar qué datos están siendo almacenados o recordados por la memoria del sistema en un punto particular de la secuencia. De igual manera, la liberación de memoria también es indicada a través de notas, generalmente en el último mensaje de un DSS dado.

2.3.1. Contratos

Un contrato de software especifica el comportamiento o efecto de una operación. En estos se describen las propiedades del resultado en lugar de dar un conjunto de pasos o instrucciones que indiquen cómo calcularlo.

Se les llama contratos pues, el consumidor se compromete a satisfacer las precondiciones necesarias al invocar una operación dada, si lo hace, tiene derecho a exigir que las postcondiciones se satisfagan, en caso de no hacerlo no se garantiza la correctitud del resultado de la invocación, se lanzan excepciones.

Para una operación de sistema “X” dada, tendremos $\{P\}S\{Q\}$, donde:

- P es la precondición de X (especificada).
- S es el programa que implementa X (a ser diseñado más adelante en la etapa de Diseño)
- Q es la postcondición de X (especificada).

En términos generales, estas condiciones refieren al estado del sistema antes y después de la invocación de la operación.

A continuación, la estructura general de un Contrato:

Nombre	Nombre del Contrato
Operación	mensaje(parámetros) : TipoDeRetorno
Entrada	Descripción de los parámetros de entrada de la operación.
Salida	Descripción de la salida de la operación.
Descripción	Descripción de que es lo que hace la operación.
Excepciones	“Si no se cumple Pre1 se lanza <i>NombreDeExcepción</i> ”

Precondiciones y Postcondiciones
Pre1: Descripción de las condiciones sobre los parámetros de la operación. Pre2: Descripción de las condiciones sobre el estado del sistema. ... Post1: Descripción de las modificaciones ocurridas en el sistema. Post2: Descripción del resultado de la operación.

En un diagrama de secuencia dado, se realizan tantos contratos como operaciones del sistema éste tenga.

3. Diseño

El diseño de software es una fase crucial en el desarrollo de sistemas, donde se transforma la especificación de requisitos en una solución funcional y estructurada. Durante esta etapa, los conceptos abstractos se concretan en componentes detallados que colaboran entre sí para cumplir con los objetivos del sistema. El diseño no solo se enfoca en cómo funciona el sistema, sino también en cómo los distintos elementos se comunican y cooperan para lograr una funcionalidad cohesiva y eficiente.

En un enfoque iterativo e incremental, el diseño de software se realiza en múltiples ciclos, refinando continuamente los componentes y sus interacciones. Este método permite adaptar el diseño a medida que se descubren nuevos requisitos o se reevalúan los existentes, asegurando así la flexibilidad y la capacidad de respuesta del sistema a futuros cambios.

3.1. Colaboración

Dentro del diseño de software, uno de los conceptos fundamentales es la colaboración. Una colaboración se define como la combinación de estructura e interacciones necesarias para llevar a cabo una operación específica o un caso de uso en el sistema. Comprende los siguientes elementos:

- **Estructura:** Incluye las clases de objetos que participan en la colaboración, junto con sus atributos y relaciones. Esta estructura proporciona el esqueleto sobre el cual se construyen las interacciones.
- **Interacciones:** Describen cómo los objetos se comunican entre sí a través de mensajes para realizar una tarea específica. Las interacciones detallan el flujo de información y control entre los componentes de la estructura.

Las colaboraciones son esenciales para diseñar soluciones efectivas y coherentes en un sistema de software al definir claramente cómo los diferentes elementos interactúan para cumplir con los requisitos del sistema.

Para diseñar y visualizar colaboraciones, se utilizan varias herramientas y diagramas en la fase de diseño:

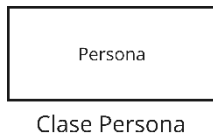
- **Diagramas de Comunicación** (ver: [Diagramas de Comunicación](#))
- **Diagramas de Clases de Diseño** (ver: [DCD](#))

3.2. Diagramas de Comunicación

Los Diagramas de Comunicación ilustran las interacciones específicas entre los objetos y clases de un sistema dado, detallando los mensajes que se envían y reciben para llevar a cabo una operación del sistema, reflejando directamente el flujo de información mediante el uso de mensajes.

A continuación, veremos toda la sintaxis para comenzar a construir Diagramas de Comunicación.

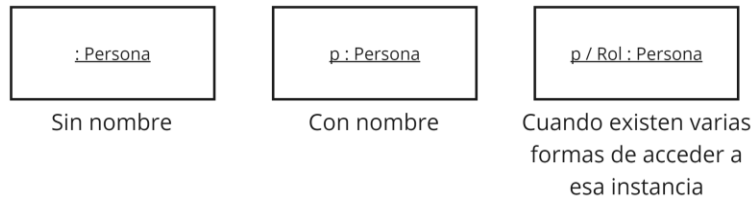
3.2.1. Clases



Las clases se representan con el nombre de la clase dentro de un rectángulo. Corresponden a la propia clase y no a una instancia particular.

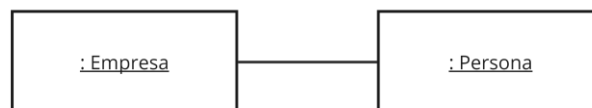
3.2.2. Instancias

Dada una clase cualquiera, por ejemplo “Persona”, una instancia de dicha clase se representa de manera similar que a una clase, tal que:



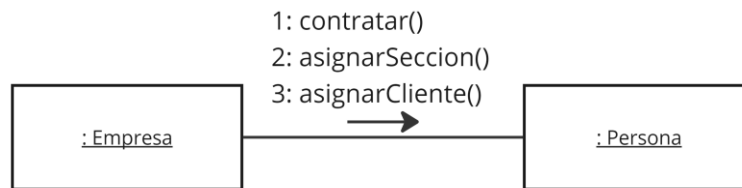
3.2.3. Links

Un link representa una conexión entre instancias que indica navegabilidad y visibilidad, estableciendo una relación cliente/servidor entre ellas.



3.2.4. Mensajes

Los mensajes son representados mediante una flecha etiquetada; un mensaje está asociado a un link y tiene asignado un número de secuencia que determina el orden de ocurrencia.



3.2.4.1. Sintaxis de los Mensajes

La sintaxis de los mensajes es la siguiente:

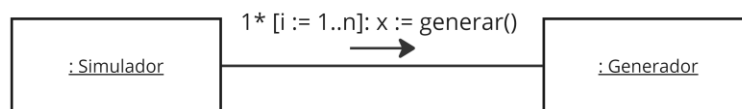
nroSec: [ret :=] mensaje([parametro [: TipoParametro]]) [: TipoRetorno]

Donde:

- **nroSec:** el numero de secuencia correspondiente al mensaje.
- **ret:** almacena el resultado de la operación (opcional).
- **mensaje:** es el nombre del mensaje enviado (y de la operación invocada).
- **parametro:** son los argumentos usados en el envío.
- **TipoParametro:** es el tipo de cada parámetro (opcional).
- **TipoRetorno:** es el tipo del recorrido de la operación (opcional).

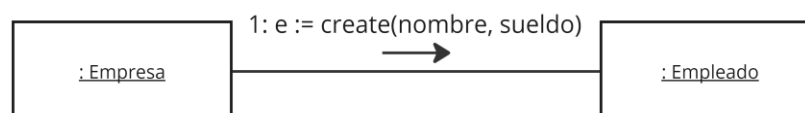
3.2.5. Iteración

Las iteraciones se indican mediante el uso de un asterisco (*) a continuación del numero de secuencia del mensaje. Esto expresa que el mensaje es enviado en forma repetida al receptor.



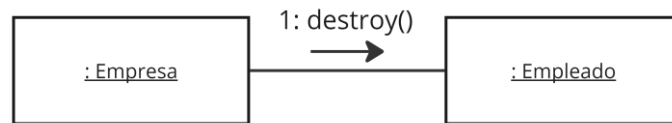
3.2.6. Creación de Instancias

La forma de ilustrar la creación de una instancia es a través del mensaje **create()** el cual es usualmente acompañado de los parámetros necesarios para la creación así como un nombre para la instancia que se crea, lo que nos permite poder utilizarla luego de creada.



3.2.7. Destrucción de Instancias

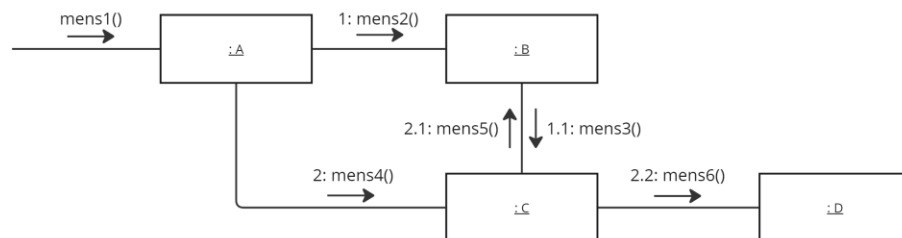
La forma de ilustrar explícitamente la destrucción de una instancia es enviando el mensaje **destroy()** teniendo el cuidado de, previo al envío del mensaje, eliminar todo link que exista con la instancia a eliminar.



3.2.8. Números de Secuencia

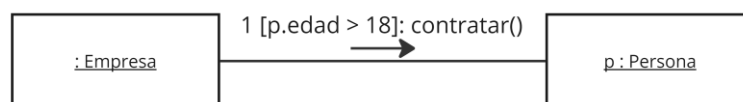
Los números de secuencia indican el orden de ocurrencia de los mensajes.

Observar que el mensaje que inicia la interacción generalmente no es numerado.



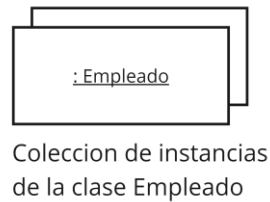
3.2.9. Mensajes Condicionales

Un mensaje condicional es enviado únicamente si su guarda, que indica la condición, es satisfecha. La misma se indica entre paréntesis rectos ([]) a la izquierda del mensaje.



3.2.10. Colecciones

Los multiobjetos de los diagramas de interacción representan una colección de objetos de una cierta clase.

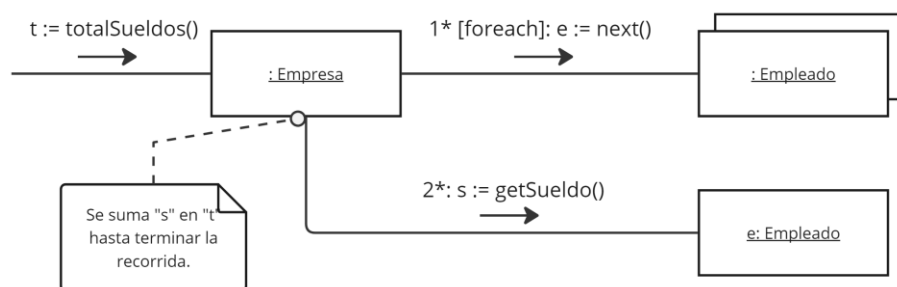


3.2.10.1. Responsabilidad de las Colecciones

Es posible comunicarse con colecciones, teniendo el cuidado que un mensaje a una colección no implica que se envía ese mensaje a cada elemento de la colección, sino que nos comunicamos con colecciones tratándolas como estructuras de datos, por lo que no tendrán mayor responsabilidad que esa, proveyendo solamente operaciones que permitan administrar los objetos contenidos.

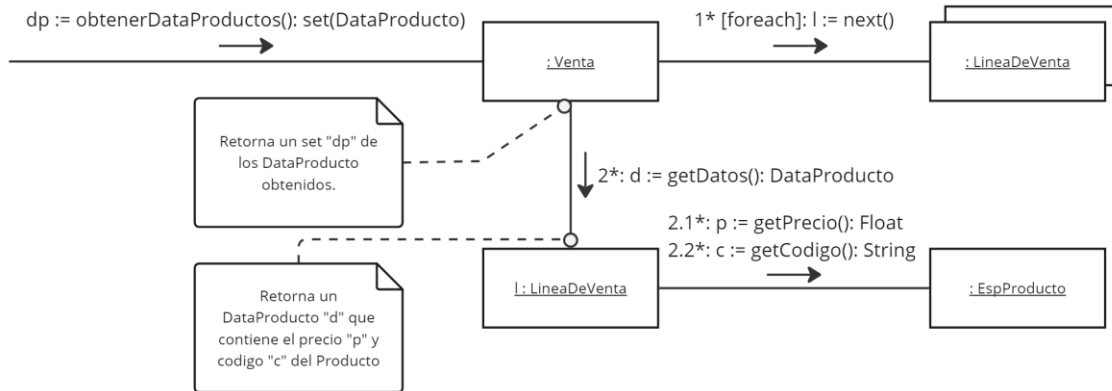
Las operaciones más comúnmente utilizadas son:

- **add(o: Tipo):** Agrega la instancia **o** a la colección.
- **remove(o: Tipo):** Remueve la instancia **o** de la colección (No elimina la instancia).
- **find(c: Clave): Tipo:** Retorna la instancia con clave **c** de tipo **Clave**.
- **exists(c: Clave): Bool:** Devuelve un booleano indicando si la instancia con clave **c** existe o no en la colección.
- **member(o: Tipo): Bool:** Devuelve un booleano indicando si la instancia **o** existe o no en la colección.
- **next(): Tipo:** Devuelve el próximo elemento en la colección.



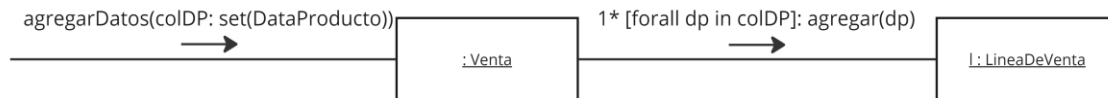
3.2.11. Datatypes

El procesamiento de datatypes (construcción, envío de mensajes, etc) no se muestra gráficamente, se utilizan notas.

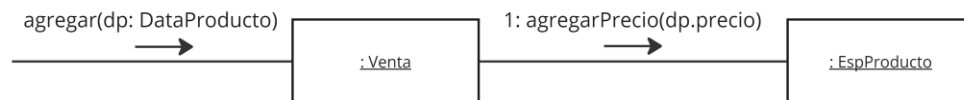


Asimismo, es posible iterar sobre los elementos de una colección de datatypes:

forall dt in ColDT



Por otro lado, es posible acceder a los elementos de un datatype utilizando el operador “.”



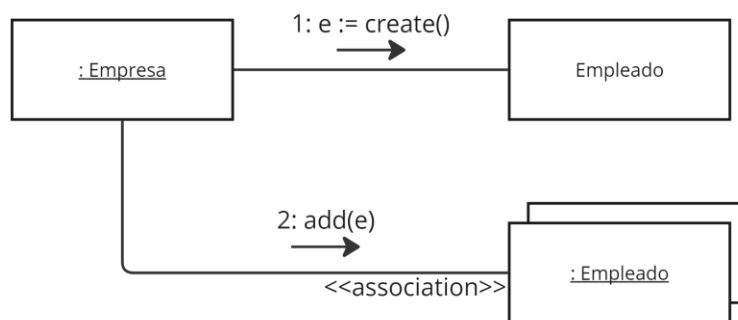
3.2.12. Visibilidad

Tal como venimos viendo, los Diagramas de Comunicación ilustran el envío de mensajes entre objetos, para que esto pueda ocurrir, un objeto dado tiene que tener visibilidad sobre el segundo, donde la visibilidad es la capacidad de un objeto de tener una referencia a otro.

Obs.: Al diseñar las visibilidades y links en un Diagrama de Comunicación, se suelen tener en cuenta los criterios GRASP (ver: [GRASP](#))

3.2.12.1. Association

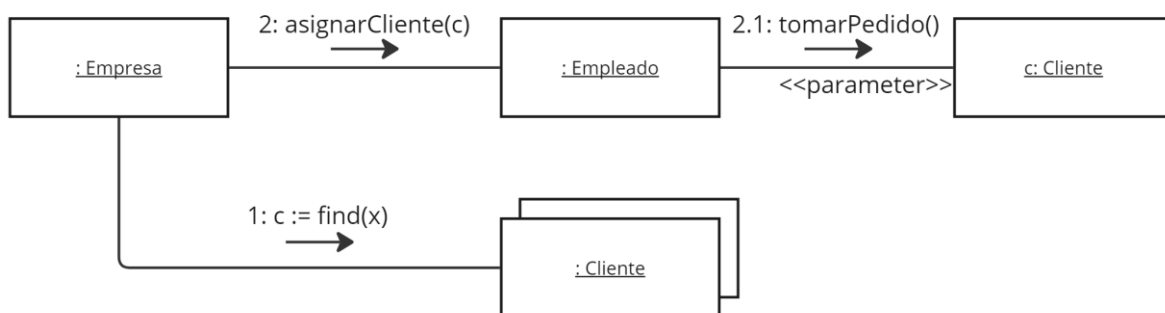
Si existe esta visibilidad entre A y B entonces B es un pseudoatributo de A. Es una visibilidad permanente ya que existe mientras que A y B existan.



La colección de empleados es un pseudoatributo de la empresa

3.2.12.2. Parameter

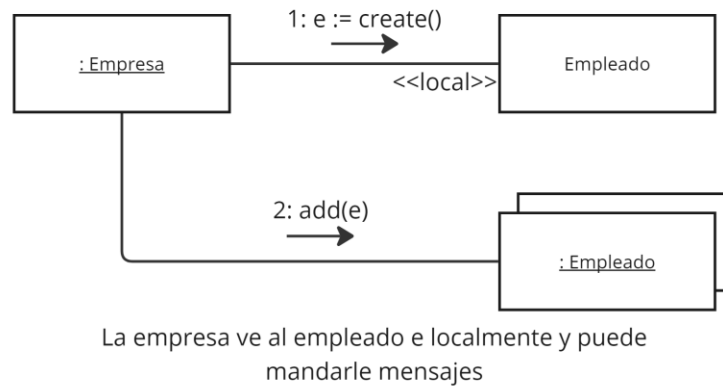
Si existe esta visibilidad entre A y B entonces B fue recibido como parámetro en un método de A. Es una visibilidad temporal, ya que existe solamente en el alcance del método.



El cliente c es recibido como parámetro por el empleado

3.2.12.3. Local

Si existe esta visibilidad entre A y B entonces B fue declarado como un objeto local en un método de A. Es una visibilidad temporal ya que existe en el alcance del método.



3.2.12.4. Global

Si existe esta visibilidad entre A y B entonces B es global a A. Es una visibilidad relativamente permanente ya que existe mientras A y B existan.

Es una forma de visibilidad poco común, de todos modos, una forma frecuente de obtenerla es mediante el patrón Singleton (ver: [Singleton](#)).

3.2.13. Criterios GRASP

Un sistema orientado a objetos está compuesto de objetos que envían mensajes a otros objetos para realizar operaciones. Para una misma operación es posible diseñar interacciones asignando responsabilidades de diferentes formas, pero la calidad del producto resultante no es la misma en todos los casos. Malas elecciones pueden conducir a sistemas que sean frágiles y difíciles de mantener, comprender, reutilizar y extender.

Para evitar estos problemas, existen criterios para la asignación de responsabilidades que nos guían hacia el diseño de una buena solución, estos son, los criterios GRASP (Object oriented design *General Responsibility Assignment Software Patterns*).

Una responsabilidad es una obligación que un objeto tiene dentro del diseño de un sistema, comprendida en términos de comportamiento. Existen dos tipos básicos de responsabilidades:

- Responsabilidad de saber o conocer: Las responsabilidades de saber incluyen conocer datos privados, conocer a otros objetos y saber cosas que pueda derivar o calcular. Por ejemplo, una transacción de un cajero automático es responsable de conocer su fecha de realización.
- Responsabilidad de hacer: Las responsabilidades de hacer incluyen hacer algo por sí mismo, iniciar acciones en otros objetos y controlar actividades de otros objetos. Por ejemplo, una transacción de un cajero automático es responsable de imprimirse a sí misma.

Las responsabilidades se asignan a clases, y sus instancias deben cumplirlas. Esto puede implicar delegar tareas a otros objetos, requiriendo interacción entre ellos. Los diagramas de comunicación muestran estas interacciones y asignaciones de responsabilidades, facilitando la implementación del diseño sin ser pseudocódigo. Los criterios GRASP guían esta asignación para lograr un diseño robusto y mantenible.

3.2.13.1. Expert

Ayuda a asignar la responsabilidad al experto en información: la clase que tiene o conoce la información necesaria para cumplir con la responsabilidad.

Para asignar esta responsabilidad hay que determinar con precisión que información se requiere y donde se encuentra la misma.

3.2.13.2. Creator

Ayuda a asignar la responsabilidad a la clase B de crear una instancia de una clase A en alguno de los siguientes casos:

- A está agregado en B
- A está contenido en B
- B registra instancias de A
- B utiliza objetos de A en forma “exclusiva”
- B es experto en crear instancias de A

Crear instancias es una de las acciones mas comunes en un sistema orientado a objetos. Es de gran utilidad disponer de un criterio general para la asignación de la responsabilidad de crear instancias.

3.2.13.3. Bajo Acoplamiento

Ayuda a asignar la responsabilidad de forma tal que el acoplamiento general se mantenga bajo, es decir, que se mantenga bajo el nivel de relacionamiento entre clases.

Una clase con bajo acoplamiento depende de pocas clases, mientras que una con alto acoplamiento depende de demasiadas clases, esto no es deseable pues:

- Cambios en las clases en las que se depende fuerzan cambios locales.
- Es difícil de comprender de manera aislada.
- Es difícil de reutilizar ya que requiere de la presencia de las clases de las que depende.

Tipos comunes de acoplamiento entre elementos X e Y pueden ser:

- X tiene un atributo de tipo Y.
- X tiene un método que referencia a una instancia de Y.
- X es subclase directa o indirecta de Y.
- Y es una interfaz y X la implementa.

3.2.13.4. Alta Cohesión

Ayuda a asignar responsabilidades de forma tal que la cohesión general se mantenga alta, es decir, se mantenga alta la medida en que tan relacionadas están entre sí las responsabilidades de una misma clase.

Una clase altamente cohesiva tiene un conjunto de responsabilidades relacionadas y no realiza una gran cantidad de trabajo, mientras que una clase con baja cohesión no es deseable ya que presenta varios problemas:

- Es difícil de comprender.
- Es difícil de reutilizar.
- Es difícil de mantener.
- Se ve afectada por cambios en forma constante.

Una clase con baja cohesión tomó demasiadas responsabilidades que pudieron haber sido delegadas a otra clase.

3.2.13.5. No Hables con Extraños

Ayuda a asignar responsabilidades de forma tal que un objeto desde un método le envíe mensajes solamente a:

- El mismo (this/self).
- Un parámetro de un método.
- Un atributo de this o self.
- Un objeto contenido en una colección que sea un atributo de this o self.
- Un objeto local.
- Un objeto global.

Se busca evitar que un objeto gane visibilidad temporalmente sobre un objeto indirecto, es decir, un objeto que no está conectado directamente a este o un objeto que está separado por un tercer objeto intermedio que está conectado directamente a ambos.

Obs.: Existen situaciones en las cuales es preferible no tener en cuenta este criterio.

3.2.13.6. Controller

Ayuda a asignar la responsabilidad de manejar las operaciones del sistema a una clase que represente una de las siguientes opciones:

- Façade Controller: existe un único controlador que provee todas las operaciones del sistema.
- Use-case Controller: existen tantos controladores como casos de uso.

¿En que casos conviene elegir uno o el otro? Un error común al diseñar controladores es asignarles demasiadas responsabilidades, en estos casos, el controlador presentaría baja cohesión (ver: [Cohesión](#)) y un alto acoplamiento (ver: [Acoplamiento](#)). Idealmente un controlador debería delegar trabajo a otros objetos mientras que el coordina la actividad.

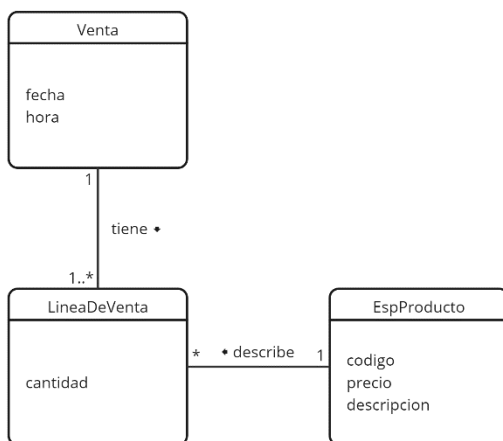
Los controladores façade son adecuados cuando se tienen pocos casos de uso y pocas operaciones del sistema en cada uno, por lo que, de no darse esta situación, un controlador façade puede verse fácilmente desbordado de responsabilidades.

Por otro lado, si se optase por una implementación con use-case controllers, en caso de contar con una cantidad de casos de uso muy grande, también se necesitaría una gran cantidad de controladores.

Ejercicio:

Vamos a ir aplicando los distintos criterios GRASP todo dentro de un mismo contexto involucrando algunas clases.

Consideremos el siguiente modelo de dominio:



¿Quién es el responsable de conocer el total de una venta? Para asignar esa responsabilidad hay que determinar que información se requiere:

- El subtotal de cada línea de la venta

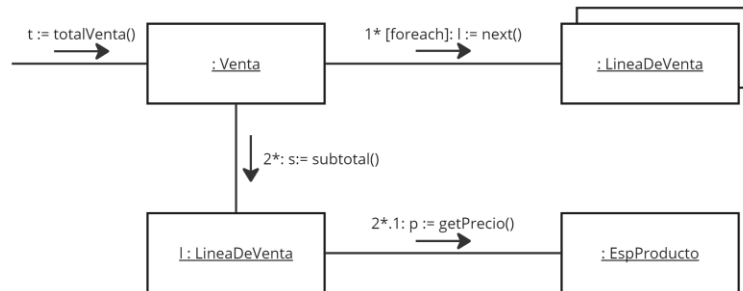
Esta información solo puede ser obtenida por la venta pues es quien conoce cada línea, luego, la clase **Venta** es la experta en conocer el total.

De igual manera, podemos preguntarnos ¿Quién es responsable de conocer el subtotal de una línea de venta?

Aplicando el mismo proceso, determinamos que la información necesaria solo puede ser obtenida por la línea de venta, pues es quien conoce la cantidad y la especificación del producto, luego, **LineaDeVenta** es experta en conocer el subtotal.

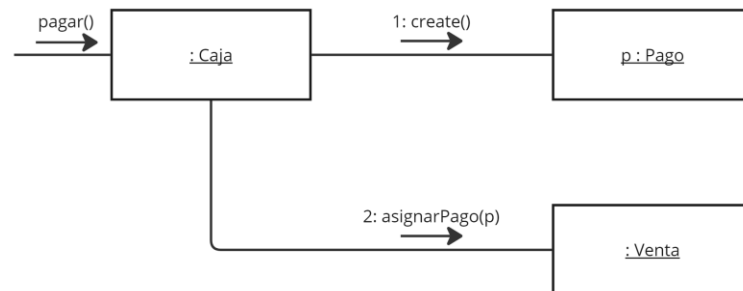
Razonando análogamente concluimos que EspProducto es la experta en conocer el precio unitario de un producto dado.

Estas asignaciones las podemos reflejar en el siguiente diagrama:

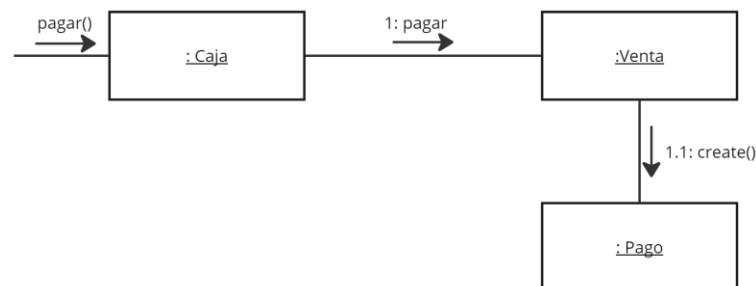


Continuando en el mismo contexto, nos preguntamos, ¿Quién es el responsable de crear instancias de LineaDeVenta?, por [Creator](#) determinamos que la clase Venta es la responsable de crear instancias de LineaDeVenta.

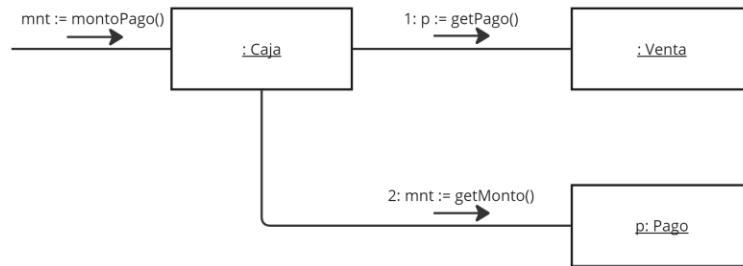
Ahora consideremos el siguiente problema: se necesita crear un pago y asociarlo a la venta correspondiente, ¿Quién es el responsable de esto?, en el mundo real una caja registradora registraría los pagos, luego, por [Creator](#) la clase Caja es candidata para ser responsable de crear los pagos. Una asignación tal produciría la siguiente solución:



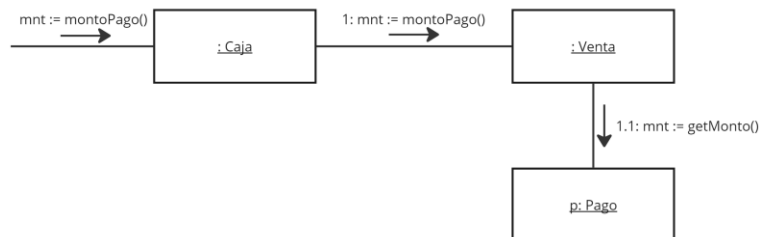
Sin embargo, aquí se acopla a Caja con Pago y a Venta con Pago. Ya que la venta termina acoplada a Pago, por [Bajo Acoplamiento](#), podríamos responsabilizar a Venta de la creación de Pago, así Caja deja de estar acoplada con Pago.



Supongamos ahora que la caja deba responder el monto de un pago dado, una solución podría ser la siguiente:



Observar que en el mensaje 2, la caja habla con un objeto indirecto, un extraño, un enfoque mas adecuado seria que la venta, en lugar de devolver el pago completo, devuelva la información del pago que la caja necesita.



Con la anterior implementación, la caja ya no habla con un extraño.

3.3. Programación en Capas

Un aspecto fundamental del diseño de software es la programación en capas, una técnica que organiza el sistema en diferentes niveles de abstracción, cada uno de los cuales tiene responsabilidades específicas. Esta aproximación facilita la separación de preocupaciones, permitiendo que cada capa se enfoque en una parte particular del sistema, mejorando así la mantenibilidad y escalabilidad.

La programación en capas divide el sistema típicamente en las siguientes capas:

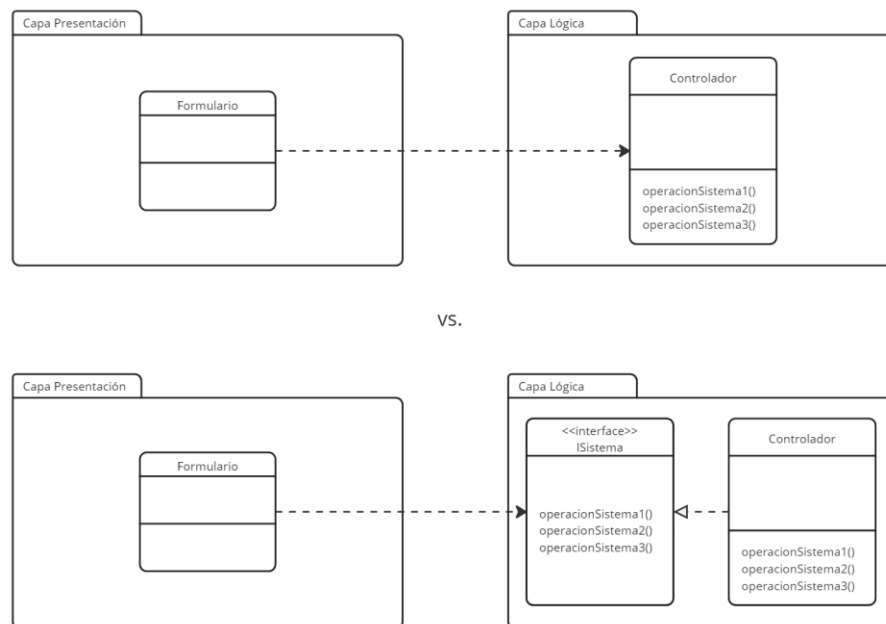
- **Capa de Presentación:** Maneja la interfaz de usuario y la interacción con el usuario.
- **Capa de Lógica de Negocio:** Contiene la lógica central del sistema, incluyendo reglas de negocio y procesos.
- **Capa de Acceso a Datos:** Gestiona la comunicación con bases de datos y otros servicios de almacenamiento.

3.3.1. Interfaces del Sistema

Las operaciones del sistema deben ser ofrecidas en interfaces las cuales son utilizadas por habitantes de la capa de presentación y son realizadas por controladores en la capa lógica.

El principal objetivo de las interfaces del sistema es quebrar la dependencia entre los elementos de la capa de presentación que invocan operaciones del sistema y los controladores de la capa lógica que las implementan.

Obs.: Usualmente cada controlador realiza una única interfaz del sistema (relación 1:1).



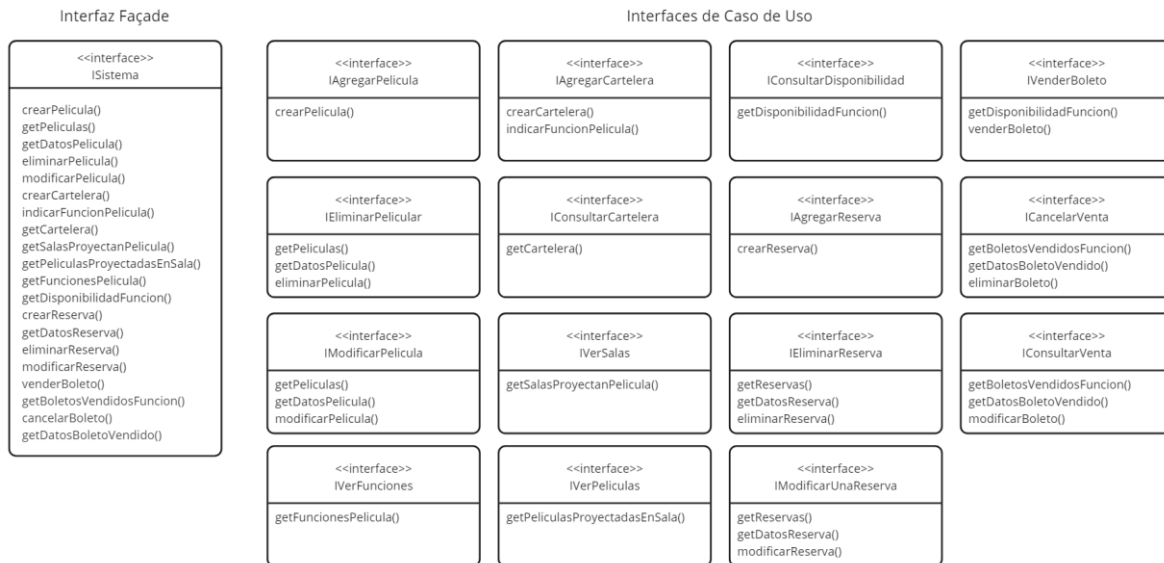
El criterio para organizar estas interfaces es el mismo propuesto por Controller (ver: [Controller](#)), con la modificación de que, cuando se tienen pocas operaciones del sistema por caso de uso, pero se tienen varios casos de uso, puede resultar conveniente optar por una solución intermedia a la propuesta por Controller.

Bajo estas circunstancias:

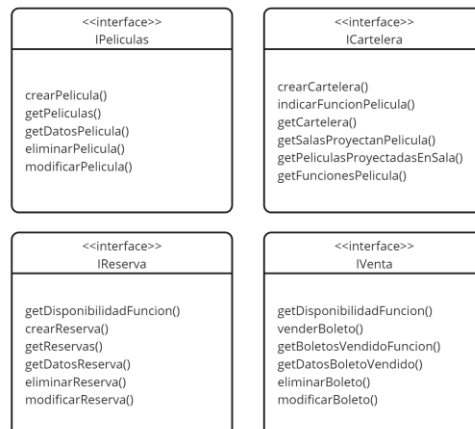
- Tener una interfaz para un controlador façade puede hacer que quede una sola interfaz demasiado grande.
- Definir una interfaz por cada caso de uso para definir controladores de caso de uso puede hacer que queden demasiadas interfaces pequeñas.

Luego, la propuesta es agrupar casos de uso que estén relacionados entre sí temáticamente y luego, definir un controlador tipo façade para cada uno de los grupos de casos de uso. De esta forma queda un “mini façade” equilibrando las interfaces y la cantidad de operaciones de sistema por interfaz.

Ejemplo:



Propuesta Intermedia

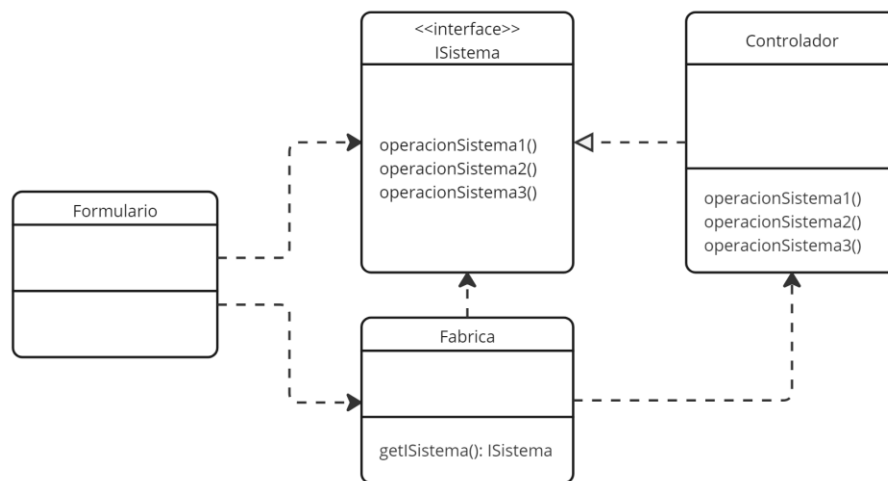


3.3.2. Fabricas

Las interfaces del sistema se definieron como un mecanismo que permite quebrar la dependencia entre las clases de presentación hacia los controladores de la capa lógica, de todos modos, definir una interfaz no es suficiente para quebrar esta dependencia.

El problema es que al momento de inicializar un Formulario se menciona explícitamente a la clase Controlador, para solucionar esto, es necesario encontrar otra forma de inicializarlo que evite mencionar a la clase Controlador, la forma de hacer esto es mediante una “fábrica” de objetos.

Una fábrica es un objeto que tiene la responsabilidad de crear instancias que realicen una interfaz determinada.



En caso de existir más interfaces del sistema, la misma Fabrica puede encargarse de devolver instancias que las realicen.

3.4. Diagrama de Clases de Diseño (DCD)

Los Diagramas de Clases de Diseño especifican la estructura detallada necesaria para soportar las interacciones definidas, incluyen clases, atributos, operaciones y relaciones, proporcionando una visión completa de la estructura del sistema.

Una vez finalizado el proceso de asignación de responsabilidades (ver: [GRASP](#)) y la construcción de todos los diagramas de comunicación necesarios, es posible especificar la parte estructural de la colaboración. Esta especificación se realiza mediante los Diagramas de Clases de UML los cuales surgen a partir de la información de diseño previa, como por ejemplo, operaciones y navegabilidades.

A continuación, se listan los elementos que se incluyen en un DCD:

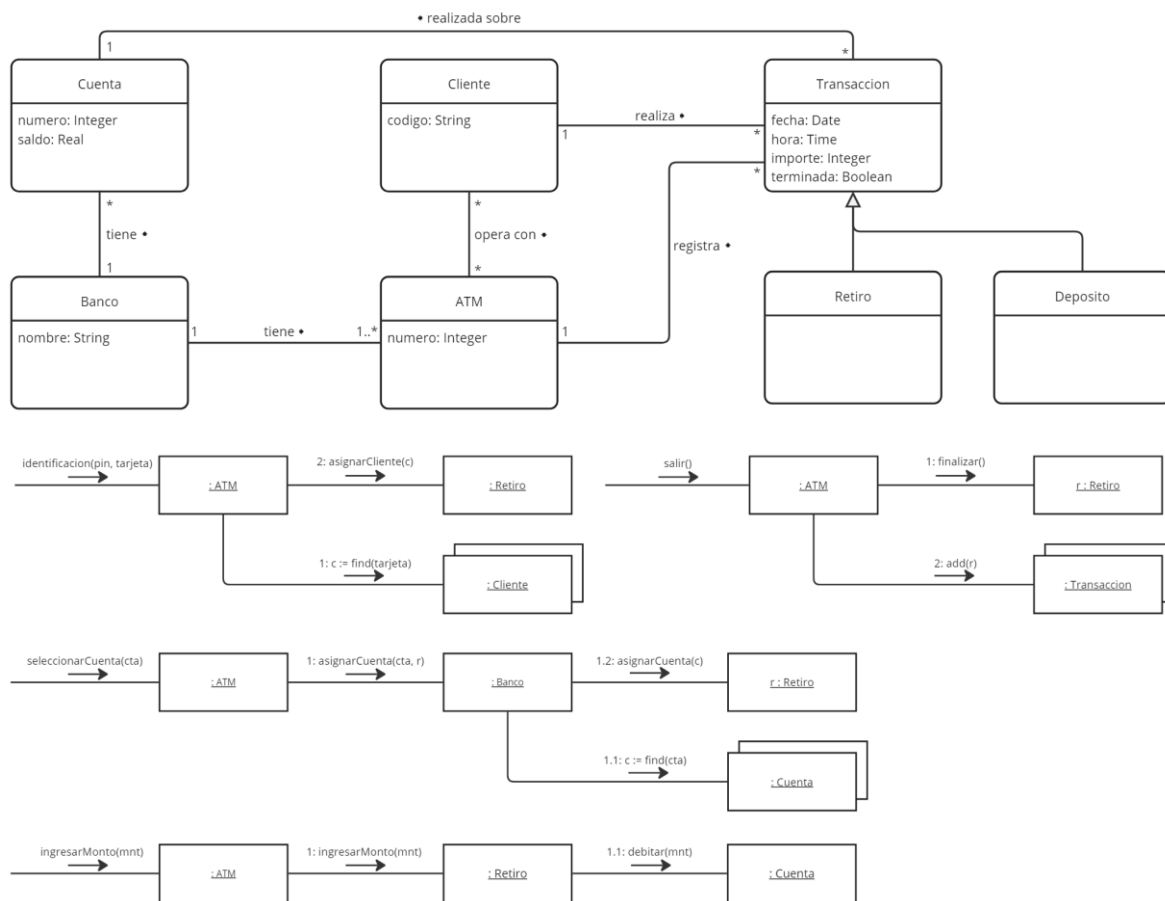
- Clases, asociaciones y atributos.
- Navegabilidades de asociaciones.
- Operaciones de clases y existencia de métodos.
- Interfaces con sus operaciones.
- Información acerca del tipo de los atributos y de los valores devueltos por las operaciones.
- Generalizaciones entre clases o interfaces.
- Dependencias entre elementos.

Sistemáticamente, a partir del dominio del problema (ver: [Modelo de Dominio](#)) y de los Diagramas de Comunicación (ver: [Diagramas de Comunicación](#)), podemos construir un DCD siguiendo los siguientes pasos:

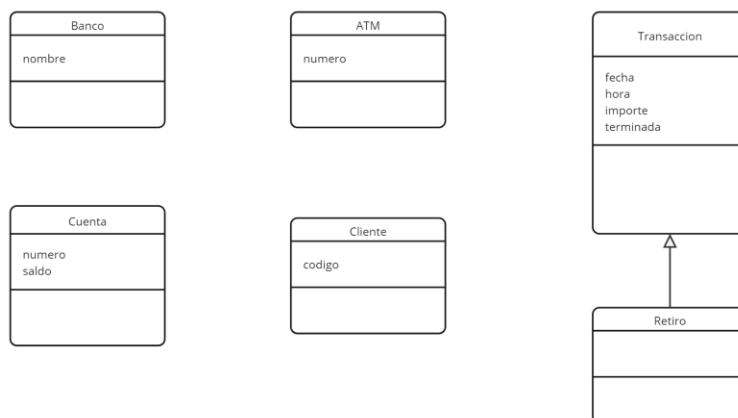
1. Identificar todas las clases que participan de la solución de los casos de uso. Hacer esto analizando los diagramas de comunicación.
2. Incluirlas en el diagrama de clases.
3. Replicar los atributos de los conceptos correspondientes en el Modelo de Dominio, agregando aquellos nuevos que sean necesarios.
4. Agregar las operaciones correspondientes a cada clase analizando los diagramas de comunicación.
5. Agregar la información de tipos a los atributos y operaciones.
6. Agregar las asociaciones necesarias para permitir las visibilidades por atributo requeridas en los diagramas de comunicación.
7. Agregar navegabilidades para indicar la dirección de cada visibilidad por atributo.
8. Agregar dependencias para reflejar los demás tipos de visibilidades existentes.
9. Agregar interfaces, fábricas y datatypes.

Ejercicio:

Consideremos el siguiente Modelo de Dominio y Diagramas de Comunicación:



Luego, si realizamos los pasos 1, 2 y 3, tendremos algo similar al siguiente diagrama:



3.4.1. Inclusión de Operaciones

3.4.1.1. Operación de Creación

La operación **create** es utilizada para la creación de instancias, ésta forma de expresarse es propia de UML e independiente de todo lenguaje de programación, por lo que este mensaje se corresponde con los constructores de las clases y, como un constructor siempre esta presente en la definición de una clase, los omitimos en los DCD.

3.4.1.2. Operaciones de Acceso

Las operaciones de acceso son utilizadas para obtener el valor de un atributo (**get**) o para modificarlo (**set**), como por defecto se asume que un atributo cualquiera de una clase tiene su get y set correspondiente asociado, omitimos estos de los diagramas.

3.4.2. Inclusión de Colecciones

Las colecciones usualmente disponen todas de las mismas operaciones, por tal razón, su inclusión no aportaría mayor información al diagrama por lo que es común omitirlas.

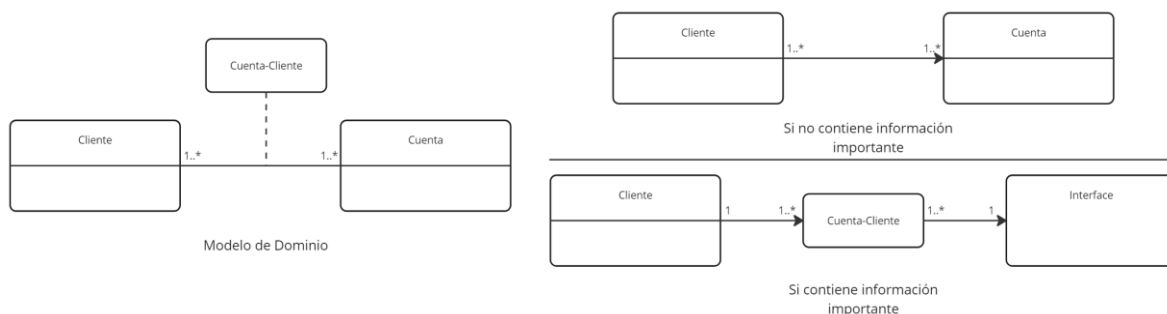
La necesidad de una colección se deriva de las multiplicidades implícitas en el diagrama.

3.4.3. Tipos Asociativos

Si repasamos los pasos para construir un Diagrama de Clases de Diseño vemos que mucha de su estructura está dada por el Modelo de Dominio, es así como nos preguntamos, ¿Cómo implementamos los tipos asociativos?

Dado que estos, al implementarse pueden contener información relevante, muchas veces es deseable mantenerlos.

Ejemplo de implementación:



3.5. Patrones de Diseño

Los Patrones de Diseño en programación son soluciones generales y reutilizables para problemas comunes que se encuentran en el diseño de software. Se pueden entender como una especie de plantillas aplicables para resolver problemas de diseño específicos en diferentes contextos.

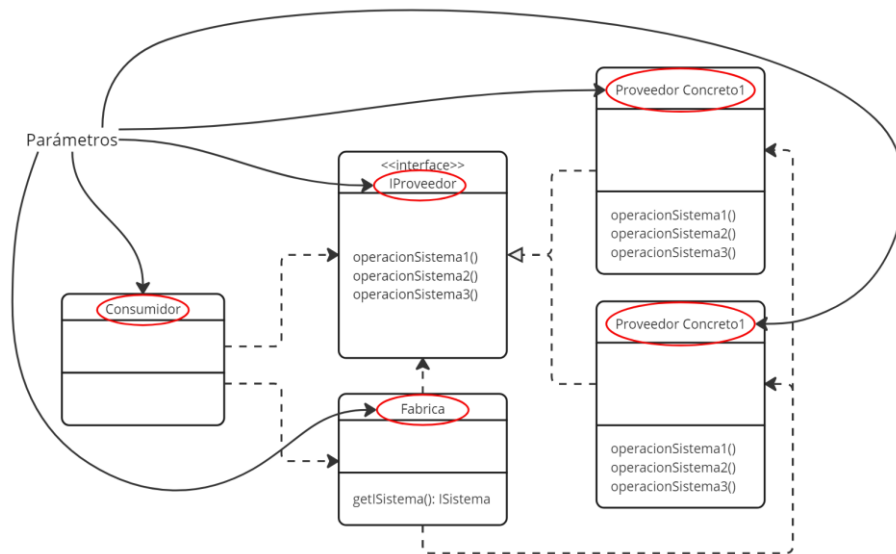
Un Patrón de Diseño sistemáticamente da un nombre, motivo y explicación de un diseño general que se aplica a un problema de diseño recurrente en sistemas orientados a objetos. Además, provee guías para su implementación y ejemplos, donde la solución es un arreglo general de objetos y clases que solucionan el problema y la solución concreta es una adaptación para un contexto particular.

3.5.1. Factory

El mecanismo de Fabricas utilizado para romper la dependencia entre la capa de presentación y la lógica (ver: [Fábricas](#)) es un Patrón de Diseño conocido como Factory.

El problema tipo que el mecanismo de Fabricas permite resolver es: “Permitir visibilidad desde un consumidor hacia proveedores concretos sin que el consumidor quede acoplado directamente a estos”.

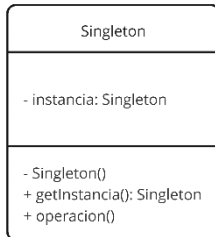
La estructura de una colaboración que solucione el problema tipo puede ser tal que:



3.5.2. Singleton

El problema tipo que el Patrón de Diseño Singleton permite resolver es: “Asegurar que una clase tenga una sola instancia y proveer un acceso global a ella”.

La estructura de una colaboración que solucione el problema tipo puede ser tal que:



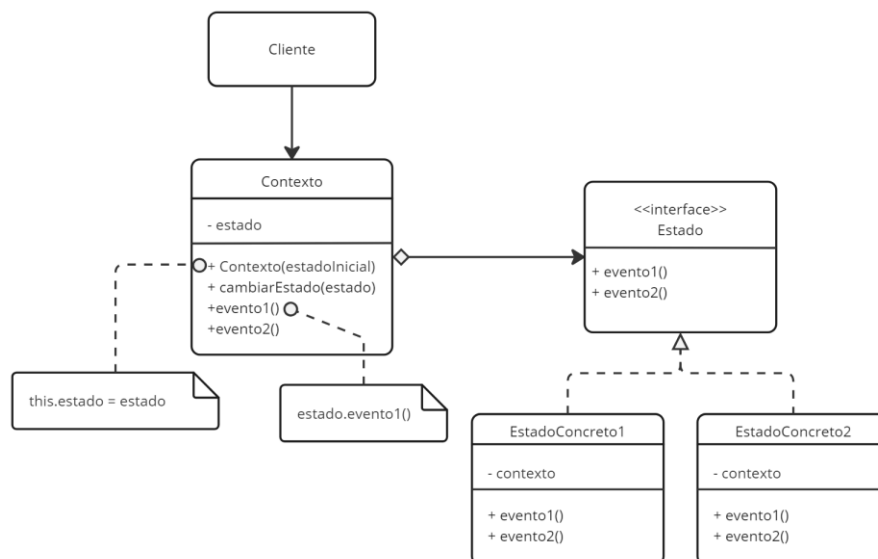
Se mantiene la instancia de clase de forma privada y se cuenta con una operación “**getInstance()**” que retorna la única instancia si ésta ya fue creada, sino, la crea y la retorna.

Una de las aplicaciones mas comunes para Singleton es su implementación en Fabricas, Controladores y Manejadores.

3.5.3. State

El problema tipo que el Patrón de Diseño State permite resolver es: “Permitir que un objeto varíe su comportamiento cuando su estado interno cambie. El objeto parecerá haber cambiado de clase”; permitiendo así cambiar el comportamiento de un objeto según su estado.

La estructura de una colaboración que solucione el problema tipo puede ser tal que:



Participantes:

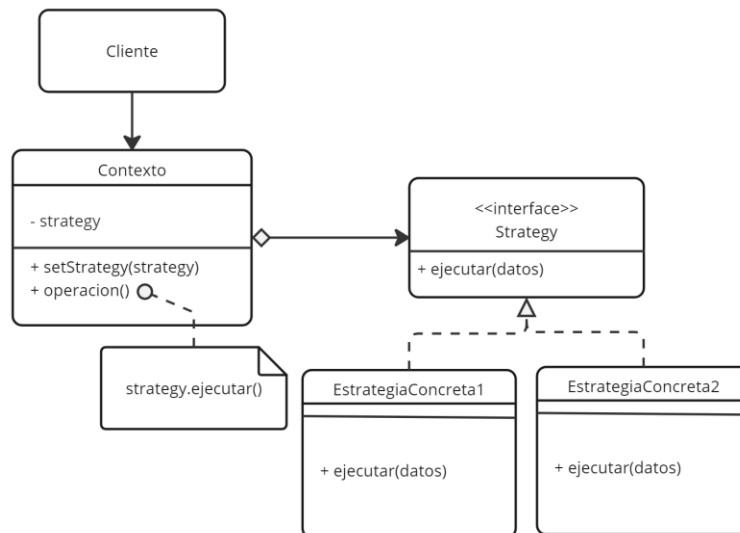
- **Contexto:** Es la clase de objetos cuyo comportamiento varía al cambiar el estado interno, en él, se mantiene una referencia a un estado concreto.
- **Estado:** Generaliza los diferentes estados concretos del Contexto.
- **EstadoConcreto:** Cada una de estas clases implementa un comportamiento particular del Contexto que sea dependiente del estado.

3.5.4. Strategy

El problema tipo que el Patrón de Diseño Strategy permite resolver es: “Definir una familia de algoritmos, encapsular cada uno de ellos y hacerlos intercambiables para que el cliente pueda elegir el algoritmo que se debe utilizar”; en otras palabras:

- Se desea definir una serie de algoritmos y permitir que los clientes seleccionen cuál utilizar en tiempo de ejecución.
- Se desea evitar el uso de condicionales largos y complejos que seleccionen el comportamiento a ejecutar.

La estructura de una colaboración que solucione el problema tipo puede ser tal que:



Participantes:

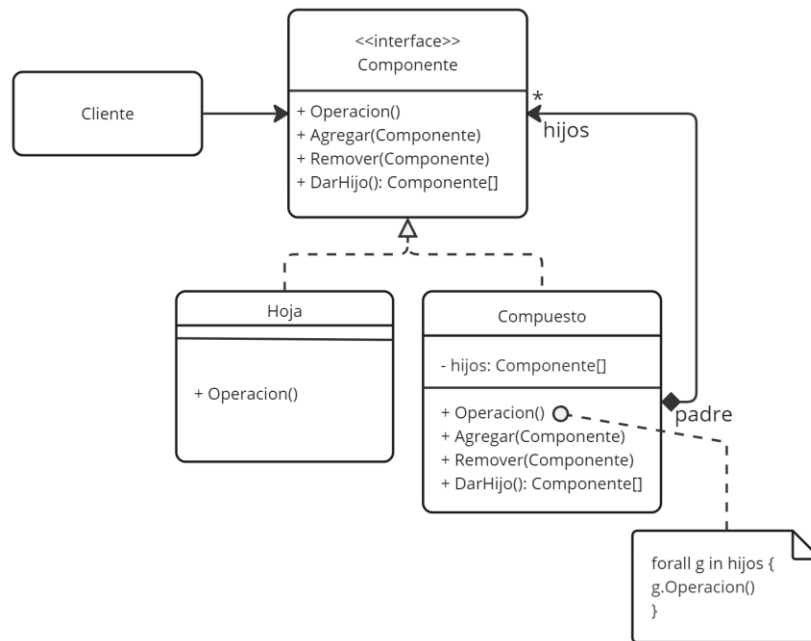
- **Contexto:** Mantiene una referencia a una **EstrategiaConcreta** y se comunica con ella a través de la interfaz. El contexto desconoce con qué tipo de estrategia trabaja o como el algoritmo es ejecutado.
- **Strategy:** Una interfaz común a todas las **EstrategiaConcreta**. Declara el método que el contexto utilizará para ejecutar una estrategia.
- **EstrategiaConcreta:** Implementa diversas variaciones del algoritmo que Contexto utiliza.
- **Cliente:** Crea o selecciona una estrategia concreta y la setea en Contexto, permitiendo al Cliente cambiar la estrategia asociada al mismo en tiempo de ejecución.

3.5.5. Composite

El problema tipo que el Patrón de Diseño Composite permite resolver es: “Componer objetos en estructuras arborescentes para representar jerarquías de objetos compuestos y tratar uniformemente los mismos”; en otras palabras:

- Se desea representar una jerarquía de objetos donde algunos de ellos se componen de otros de la misma jerarquía.
- Se desea que los clientes ignoren las diferencias entre objetos compuestos y objetos primitivos, y los tratan uniformemente.

La estructura de una colaboración que solucione el problema tipo puede ser tal que:



Participantes:

- **Componente:** Declara el conjunto de operaciones comunes a los objetos en la jerarquía e implementa el comportamiento común por defecto permitiendo acceder y manipular a los componentes hijo, y, opcionalmente, los componentes padre.
- **Hoja:** Define el comportamiento de los objetos primitivos en la composición.
- **Compuesto:** Define el comportamiento de los objetos compuestos, delegando a sus hijos, de los cuales mantiene una referencia, el comportamiento relacionado con ellos.
- **Cliente:** Manipula los objetos de la composición a través de una instancia **Componente**.

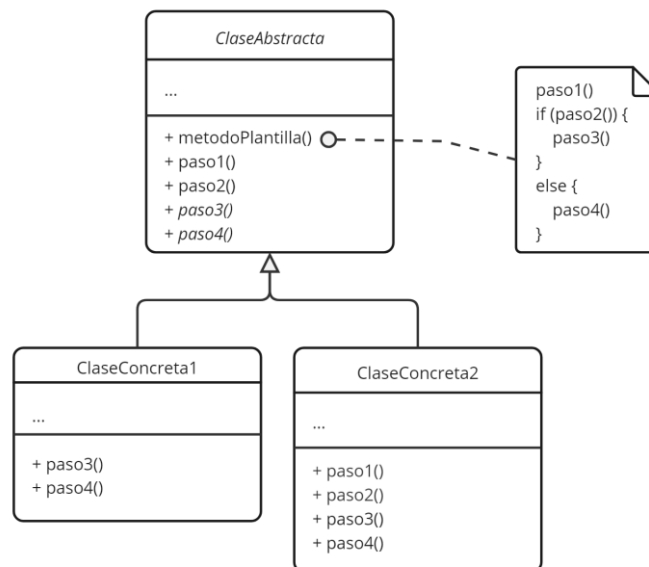
Como consecuencia, permite al cliente manipular objetos de la jerarquía sin discriminar entre objetos primitivos y objetos compuestos, facilitando el proceso de agregar nuevos componentes a la jerarquía sin modificar el código del cliente.

3.5.6. Template Method

El problema tipo que el Patrón de Diseño Template Method permite resolver es: “Definir el esqueleto de un algoritmo en una operación, permitiendo a las subclases redefinir ciertos pasos del algoritmo sin cambiar su estructura”; en otras palabras:

- Se desea que el algoritmo principal tenga una estructura fija, pero que ciertos pasos específicos puedan ser personalizados por las subclases.
- Se desea evitar la duplicación de código común en diferentes implementaciones del mismo algoritmo.
- Se desea controlar el flujo de un algoritmo, asegurando que ciertos pasos ocurran en un orden específico, mientras se permite la flexibilidad de definir detalles específicos en las subclases.

La estructura de una colaboración que solucione el problema tipo puede ser tal que:



Participantes:

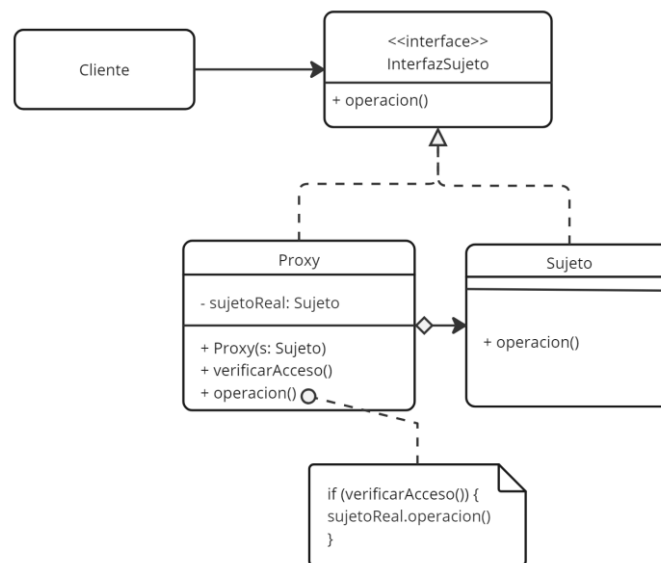
- **ClaseAbstracta:** Declara los métodos que actúan como pasos para un algoritmo, así como la propia plantilla para dicho método el cual llama a los otros métodos en un orden específico. Observar que los pasos pueden ser virtuales o tener una implementación por defecto.
- **ClaseConcreta:** Implementa o sobrescribe uno o varios métodos de **ClaseAbstracta** pero no la plantilla misma.

3.5.7. Proxy

El problema tipo que el Patrón de Diseño Proxy permite resolver es: “Proporcionar un sustituto o representante para controlar el acceso a otro objeto”; en otras palabras:

- Se desea controlar el acceso a un objeto, realizando ciertas acciones antes o después de que el objeto real sea accedido.
- Se desea añadir una capa de seguridad o validación antes de permitir el acceso al objeto real.
- Se desea retrasar la creación o la carga de un objeto costoso hasta que realmente sea necesario.

La estructura de una colaboración que solucione el problema tipo puede ser tal que:



Participantes:

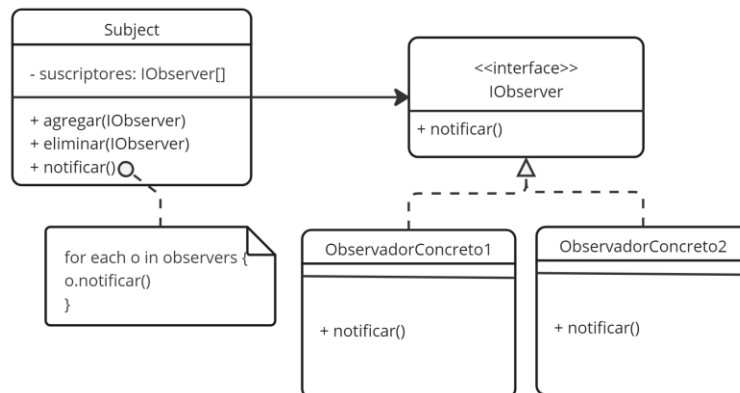
- **InterfazSujeto:** Declara una interfaz común tanto para Proxy como para Sujeto, de modo que el Proxy pueda ser utilizado donde se espera un Sujeto real.
- **Proxy:** Mantiene una referencia al Sujeto real y controla el acceso a él. Puede encargarse de tareas adicionales como la creación, inicialización, la seguridad, control de acceso, almacenamiento en caché y la sincronización.
- **Sujeto:** Define el objeto real al que el Proxy representa.

“El patrón Proxy es como tener un asistente que maneja el acceso a algo importante, como un archivo grande o una conexión complicada. Este asistente puede hacer cosas útiles antes de permitirte usarlo, como asegurarse de que tienes permiso para acceder, o cargar el archivo solo cuando realmente lo necesitas para ahorrar tiempo y recursos. En pocas palabras, el Proxy simplifica tu interacción con cosas complicadas, añadiendo funciones extras útiles sin que tengas que preocuparte por ellas directamente.”

3.5.8. Observer

El problema tipo que el Patrón de Diseño Composite permite resolver es: “Definir una dependencia 1 – n entre objetos, de forma que cuando uno cambie de estado, todos los dependientes sean notificados”; es decir, permite que un objeto notifique a otros objetos de diferente naturaleza y sin estar acoplado a ellos.

La estructura de una colaboración que solucione el problema tipo puede ser tal que:



Participantes:

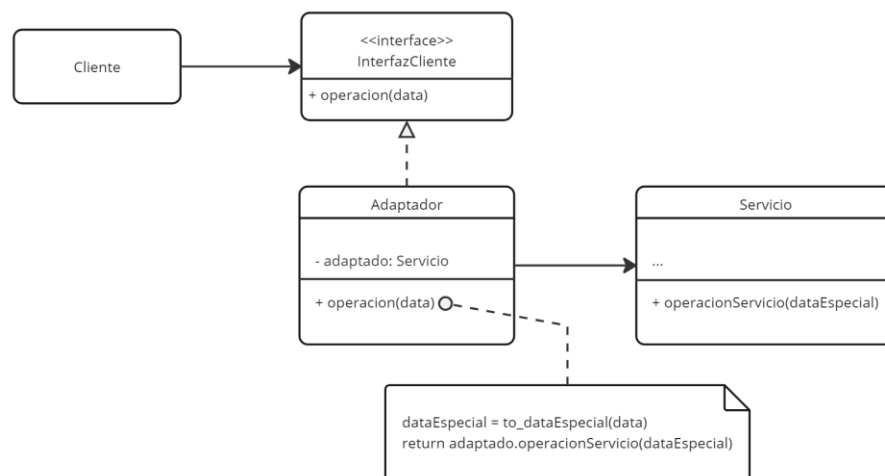
- **Subject:** Dispone de información que de interés para un conjunto de objetos interesados y los notifica cuando se considera necesario.
- **IObserver:** Declara la operación por la cual los interesados son notificados.
- **ObservadorConcreto:** Un interesado en los avisos que el Subject tenga para enviar.

3.5.9. Adapter

El problema tipo que el Patrón de Diseño Adapter permite resolver es: “Permitir que interfaces incompatibles trabajen juntas”; en otras palabras:

- Se desea utilizar una clase existente pero su interfaz no coincide con la interfaz requerida por el sistema.
- Se quiere reutilizar una clase existente que no tiene la interfaz necesaria para una nueva aplicación.
- Se necesita interactuar con múltiples clases o bibliotecas que tienen interfaces diferentes y se desea unificar su acceso a través de una interfaz común.

La estructura de una colaboración que solucione el problema tipo puede ser tal que:



Participantes:

- **Cliente:** Es una clase que contiene la lógica de negocio existente del programa.
- **InterfazCliente:** Describe un protocolo que otras clases han de seguir para poder colaborar con el código del Cliente.
- **Servicio:** Es alguna clase útil que no podemos usar directamente porque tiene una interfaz incompatible.
- **Adaptador:** Es una clase que es capaz de trabajar tanto como con el Cliente como con el Servicio implementando la InterfazCliente mientras que guarda un objeto Servicio. El adaptador recibe llamadas del Cliente a través de InterfazCliente y las traslada a llamadas al Servicio contenido en un formato que pueda entender.

4. Implementación

Una vez finalizada la etapa de Diseño (ver: [Diseño](#)), contaremos con, al menos, una colaboración, nuestro propósito ahora es implementarla, es decir, a partir de los diagramas resultantes de la etapa anterior, utilizarlos para generar código fuente funcional.

Para implementar una colaboración dada, se ha de:

1. Implementar Interfaces.
2. Implementar Clases.
 - a. Implementar atributos.
 - b. Implementar operaciones.
3. Implementar Relaciones.
 - a. Implementar generalizaciones.
 - b. Implementar realizaciones.
 - c. Implementar asociaciones.
4. Implementar interacciones.
 - a. Implementar métodos.

Obs.: Todos los ejemplos de código a continuación serán en C++ por lo que su implementación específica puede variar en otros lenguajes.

4.1. Interfaces

Una interfaz puede ser entendida como un conjunto de operaciones a la que se le aplica un nombre, éstas no asocian un método para sus operaciones y, en general, estas operaciones caracterizan el comportamiento o parte del comportamiento de las instancias de clases. Por otro lado, hallamos especial interés en las interfaces al implementar un paradigma en capas (ver: [Interfaces - Diseño](#)).

Las interfaces se implementan directamente a partir del DCD. Las operaciones que éstas ofrecen se obtienen de la propia especificación de la interfaz.

Obs.: En casos como C++, que no proveen una construcción para implementar interfaces directamente se suele implementar una clase abstracta, sin atributos y con todas sus operaciones abstractas.

Ejemplo:

```
1  class IControladorUsuario // Interfaz
2  {
3  public:
4      virtual bool login(string usuario, string password) = 0;
5      virtual void logout() = 0;
6      virtual bool actualizarPassword(string passwordAntiguo, string passwordNuevo) = 0;
7      virtual set<DataPermiso> permisos() = 0;
8      virtual set<DataUsuario> getAmigos() = 0;
9      virtual DataUsuario getPerfil() = 0;
10     // ...
11     virtual ~IControladorUsuario(){}; // virtual y vacío
12 };
```

4.1.1. Realización

Es una relación entre una especificación y su implementación. Una forma posible de realización se produce entre una interfaz y quien la implementa, sea una clase o un controlador.

Se dice que una clase C realiza una interfaz I si C implementa todas las operaciones declaradas en I, es decir, provee un método para cada una de ellas.

Ejemplo:

```
1 // Una interfaz para implementar lecturas secuenciales sobre algo
2 class Archivo {
3 public:
4     // lee hasta n caracteres, avanza hasta la última posición
5     // leída y devuelve su contenido en el string
6     virtual string read(int n) = 0;
7     // pone la última posición leída principio del archivo
8     virtual void rewind() = 0;
9     // dice en qué posición se va a leer
10    virtual int position() = 0;
11    // dice cuántos caracteres hay disponibles
12    virtual int available() = 0;
13    virtual ~Archivo() {};
14 };
```

```
1 class CPPArchivo: public Archivo
2 {
3 private:
4     fstream f;
5
6 public:
7     CPPArchivo(const string& ruta);
8     // implementación de interfaz Archivo
9     string read(int n);
10    void rewind();
11    int position();
12    int available();
13    // operaciones extra
14    void close();
15    void write(const string& s);
16    void setPosition();
17    ~CPPArchivo();
18 };
```

Obs.:

1. Lo anterior es el Header de la clase CPPArchivo, su implementación se da por separado.
2. Una clase puede realizar cualquier cantidad de interfaces.

4.2. Clases

Como vimos previamente, una clase es un descriptor de objetos que comparten los mismos atributos, operaciones, métodos, relaciones y comportamiento.

Al igual que con las interfaces, la implementación de las clases se hace en forma directa a partir del DCD. Pero, a diferencia de las interfaces, los lenguajes de programación, orientados a objetos, si ofrecen una construcción para este fin.

Los atributos y operaciones se obtienen a partir de la propia especificación de la clase, incluyendo constructor, destructor, así como operaciones de acceso y/o modificación de los atributos.

Ejemplo:

```
1  class Usuario {
2  private:
3      string nickname;
4      string password;
5      DataFecha fechaNacimiento; // Datatype
6  public:
7      // Constructores:
8      Usuario();
9      Usuario(string, string, DataFecha);
10
11     // Getters:
12     string getNickname();
13     string getPassword();
14     DataFecha getFechaNacimiento();
15
16     // Setters:
17     void setNickname(string);
18     void setPassword(string);
19     void setFecha(DataFecha);
20
21     // Ops:
22     //...//
23     virtual ~Usuario() = 0; // Destructor
24 };
```

4.3. Relaciones

4.3.1. Generalizaciones

Las generalizaciones se obtienen directamente del DCD.

Los lenguajes de programación orientados a objetos proveen una construcción para esto. En la declaración de la clase se especifica su ancestro. Muchos lenguajes de programación permiten un único ancestro, C++ permite varios.

Ejemplo:

```
1  class Jornalero: public Empleado {  
2      //...//  
3  }
```

4.3.2. Realizaciones

Las realizaciones también se obtienen directamente del DCD.

Los lenguajes de programación que no proveen interfaces tampoco proveen realizaciones, por lo que, en el caso de C++, se utiliza una generalización.

4.3.3. Asociaciones

Los lenguajes de programación no suelen proveer una construcción específica para la implementación de asociaciones.

Para que una clase A pueda estar asociada a una clase B se suele incluir un atributo en A; este atributo no pertenece al conjunto de atributos definidos en el diseño por lo que se lo denomina “pseudoatributo”. A través del pseudoatributo, una instancia de A puede mantener una referencia a otra de B y así implementar el link.

Se define un pseudoatributo en A solamente si la asociación es navegable hacia B. El tipo del pseudoatributo para A depende de la clase B, pero también la multiplicidad en el extremo de la asociación del lado de B. Distinguimos casos según el máximo de dicha multiplicidad:

- Si el máximo es 1:
 - o El pseudoatributo es de tipo **B***
- En cualquier otro caso:
 - o El pseudoatributo es de tipo colección, puede ser **set**, **map**, **vector**, etc (**ICollection** en general) dependiendo de las necesidades de navegación.

4.4. Interacciones

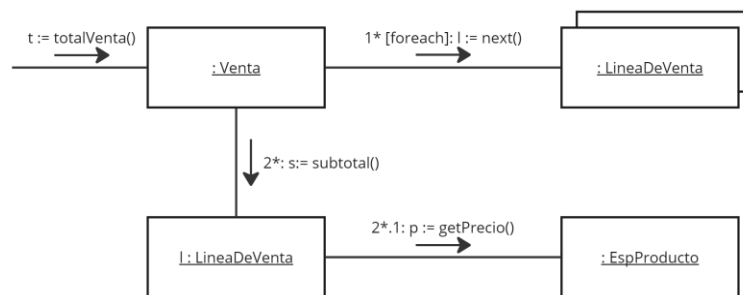
La implementación de la estructura conduce a la definición de elementos de diseño junto con sus relaciones. Hasta este punto las clases incluyen sus operaciones, pero no los métodos asociados, esto implica que no existen invocaciones implementadas y por lo tanto, no hay comportamiento. Es a partir de los diagramas de comunicación, que no tienen como objetivo servir de pseudocódigo pero que generalmente ilustran la lógica general de las operaciones, que se extrae la información necesaria para implementar los métodos.

Luego, al implementar el método asociado a la operación **op()** en una clase A:

- Se busca el diagrama de comunicación que incluya un mensaje **op()** llegando a una instancia de A.
- La interacción anidada en ese mensaje debería resultar de ayuda para implementar el método.

Ejemplo:

Consideremos el siguiente diagrama de comunicación:



Para implementar el método asociado a **totalVenta()** observamos la interacción anidada en el mensaje en el diagrama de comunicación. Luego, obtenemos la siguiente implementación:

```
1  class Venta {
2      private:
3          ICollection *lineas;
4      public:
5          float totalVenta();
6          virtual ~Venta();
7  };
8
9  float Venta::totalVenta() {
10     float total = 0;
11     for(IIterator *it = lineas->getIterator(); it->hasCurrent(); it->next()) {
12         LineaDeVenta *ldv = dynamic_cast<LineaDeVenta *>(it->getCurrent());
13         total += ldv->subtotal();
14     }
15     return total;
16 };
```

4.5. Manejo de Objetos

Los objetos son manipulados a través de referencias. Dependiendo de como los lenguajes de programación las implementen, aplican ciertas consideraciones tanto a la manipulación como a la destrucción de objetos.

En algunos lenguajes las referencias se implementan explícitamente. En particular, en C++, las referencias se implementan mediante punteros.

4.5.1. Referencias

En tiempo de ejecución los objetos no son alojados en el stack, sino en el heap. La forma de acceder a un objeto es mediante una referencia. Una referencia es una variable que es alojada en el stack que, o no identifica a ningún objeto (void), o identifica a un objeto particular de una determinada clase (attached).

Algunas de las operaciones más comunes con referencias en C++ son:

```
1 // Referencia a un objeto de clase A:
2 ClaseA *aPtr;
3
4 // Variable en el stack:
5 Clase aObj;
6
7 // Hacer que una referencia sea void:
8 aPtr = NULL;
9 bPtr = nullptr;
10
11 // Hacer que una referencia sea attached:
12 ClaseA *aPtr = new ClaseA();
13 ClaseA *bPtr = NULL;
14
15 // Adjuntar la referencia a un objeto a través de otra referencia:
16 bPtr = aPtr; // Asignación de referencias (punteros)
```


4.5.2. Copias

En sistemas orientados a objetos, es usual que un objeto sea “conocido” por otros varios objetos, es deseable que todos estos objetos referencien al mismo objeto y no a copias de él. Así es que la identidad y la necesidad de compartir objetos hace que en general no sea correcto copiar objetos, teniendo en cuenta que una copia de un objeto es otro objeto cuyas propiedades son iguales a las del original.

De todos modos, distinguimos dos formas de realizar copias de objetos:

- **Copia plana:** Su resultado es un objeto exactamente igual al original, incluyendo sus referencias.
- **Copia en profundidad:** El objeto resultante es exactamente igual al original, salvo las referencias.

A su vez, distinguimos tres casos respecto a la copia de objetos:

- **Objetos que implementan data values:** En C++ los objetos los implementamos mediante clases, por lo cual, formalmente, sus instancias serán objetos, sin embargo, carecen de identidad, ya que son data values; La copia de data values se realiza en profundidad.
- **Objetos que son instancias de clases del diseño:** Los objetos si presentan identidad, por lo que, en caso de requerir a mas de uno desde mas de un lugar, se debe compartirlo, no copiarlo.
- **Objetos que representan colecciones:** El caso de las colecciones es particular, porque pueden involucrar una estructura de datos (sin identidad) u objetos (con identidad). Las colecciones de data values se tratan como el caso de los propios data values (en profundidad). Por otro lado, la copia de colecciones de objetos se realiza en forma plana, es decir, copiamos las referencias, pero el objeto al que referencian ambas colecciones son los mismos.

4.5.3. Destrucción

Los objetos alojados en el heap permanecen allí hasta que el programa termina, a diferencia de aquellos alojados en el stack, por lo que, cuando un objeto ya no es de utilidad se lo suele retirar del heap para liberar la memoria. En C++ este proceso de liberación de memoria se realiza de forma manual ya que no se dispone de Garbage Collector.

```
1  class miClase {
2      private:
3          int id;
4      public
5          miClase(int);
6  }
7
8  miClase devuelvoMiClase() {
9      miClase *c1 = new miClase(1);
10     miClase *c2 = new miClase(2);
11     miClase *c3 = c1;
12
13     delete c1; // Liberacion de memoria manual.
14     return c3; // c3 es una referencia colgada pues se "deleteó" c1.
15 } // La referencia adjunta a c2 queda inaccesible.
```

Obs.: Hay programas que permiten detectar perdidas de memoria y referencias colgantes como valgrind. Son lentos, pero pueden resultar de utilidad.

4.6. Colecciones

Las colecciones de objetos son fundamentales para manejar asociaciones con múltiples elementos, permitiendo iterar sobre ellos, realizar búsquedas y gestionar claves. Estas colecciones comparten una estructura uniforme y reutilizable, facilitando su implementación. Existen dos tipos principales: las que contienen elementos con claves y las que no. Para optimizar su uso, se pueden definir infraestructuras comunes que sirvan de base para todas las colecciones específicas, como las colecciones paramétricas y las colecciones genéricas, que pueden almacenar cualquier tipo de elemento mediante la interfaz de marca **ICollectionable**.

4.6.1. Colecciones Genéricas

Las colecciones genéricas están diseñadas para contener cualquier tipo de elemento. La interfaz **ICollectionable** permite que las clases puedan almacenar sus objetos en estas colecciones. La especificación e implementación de estas colecciones se separan mediante la interfaz **ICollection**, permitiendo múltiples implementaciones específicas como listas, vectores y árboles binarios. Esta separación facilita la creación de diferentes estructuras de datos que pueden adaptarse a diversas necesidades.

4.6.2. Iteradores

Los iteradores son esenciales para recorrer los elementos de una colección sin exponer su estructura interna. La interfaz **Iterator** permite iterar sobre los elementos de manera secuencial y eficiente.

4.6.3. Diccionarios

Los diccionarios son colecciones especializadas donde cada elemento está asociado a una clave única, facilitando la búsqueda, inserción y eliminación de elementos. Esta estructura es ideal para implementar tablas de símbolos y otras estructuras que requieren acceso rápido a datos mediante una clave.

4.6.4. Búsquedas

La búsqueda en colecciones es una operación esencial para encontrar elementos específicos. Las colecciones deben soportar diferentes tipos de búsquedas, desde las más simples hasta las más complejas. Las estrategias de búsqueda varían según la estructura de datos subyacente, como listas, árboles binarios o tablas hash, cada una ofreciendo distintos niveles de eficiencia y complejidad.

4.6.5. Contenedores STL

Notar que en los puntos anteriores se evitó hablar de código, dar ejemplos y entrar en tecnicismos, pues en la práctica, no se implementan manualmente, sino que utilizamos bibliotecas para el manejo de colecciones que ya implementan todo esto.

C++ ofrece la Standard Template Library (STL), un conjunto de contenedores, iteradores y otras utilidades para manejar conjuntos de objetos. Algunos de los contenedores mas comunes que utilizamos en la practica son:

- **set<T>**: Una colección de objetos de tipo T. Permite, opcionalmente, ordenamiento.
- **map<K, V>**: Un Diccionario de claves de tipo K y valores de tipo V.
- Otros: **vector**, **queue**, **list**, **stack**.

4.6.5.1. Set

Set permite el almacenamiento de colecciones de elementos, permite ordenamiento, utilizando el operador "<" (muchas veces necesita sobrescribir **operator<**), y permite iterar sobre los elementos almacenados.

Algunas de las operaciones más comunes que **set** incorpora son:

- **empty()**: Devuelve True si la colección esta vacia, False en caso contrario.
- **size()**: Devuelve un entero indicando la cantidad de elementos almacenados.
- **insert(elemento)**: Agrega "elemento" a la colección donde "elemento" tiene que ser del tipo adecuado para la colección.
- **find(elemento)**: Un iterador que apunta al elemento encontrado. Si el elemento no se encuentra, el iterador es igual a **end()**.
- **erase(elemento)**: Elimina un elemento específico del conjunto siempre que este se encuentre dentro de la colección.
- **begin()**: Devuelve un iterador apuntando al primer elemento del conjunto.
- **end()**: Devuelve un iterador que apunta al final del conjunto. Entiéndase por final del conjunto no al ultimo elemento existente dentro del conjunto, sino un elemento más allá.

Ejemplo de uso:

```
1  set<Empleado *> col;
2
3  Empleado *e1 = new Empleado("Bob");
4  Empleado *e2 = new Empleado("Ana");
5  Empleado *e3 = new Empleado("Laura");
6
7  col.insert(e1);
8  col.insert(e2);
9
10 // Listar empleados, uso de Iterator:
11 for (set<Empleado *>::iterator it = col.begin(); it != col.end(); ++it) {
12     Empleado *actual = (*it);
13     cout << actual->getNombre();
14 }
15
16 col.find(e1); // Devuelve un Iterator situado en el elemento Bob.
17 col.find(e3); // Devuelve un Iterator apuntando a col.end()
```

4.6.5.2. Map

Map brinda las mismas funciones que **IDictionary**, permitiendo realizar búsquedas por clave, así como iteraciones. Asimismo, permite cambiar la forma de comparación de claves.

Algunas de las operaciones más comunes que **map** incorpora son:

- **empty()**: Devuelve True si la colección esta vacia, False en caso contrario.
- **size()**: Devuelve un entero indicando la cantidad de elementos almacenados.
- **operator[](clave)**: Devuelve una referencia al valor asociado con la clave dada. Si la clave no existe, devuelve NULL.
- **find(clave)**: Un iterador que apunta al elemento encontrado con la clave dada. Si el elemento no se encuentra, el iterador es igual a **end()**.
- **erase(clave)**: Elimina un elemento específico con la clave dada del conjunto siempre que este se encuentre dentro de la colección.
- **begin()**: Devuelve un iterador apuntando al primer elemento del conjunto.
- **end()**: Devuelve un iterador que apunta al final del conjunto. Entiéndase por final del conjunto no al último elemento existente dentro del conjunto, sino un elemento más allá.

Ejemplo de uso:

```
1  map<string, Usuario *> col;
2
3  Usuario *u1 = new Usuario("Ana", 22);
4  Usuario *u2 = new Usuario("Bob", 31);
5
6  // Agrego los usuarios al map, dos formas:
7  col["Ana"] = u1;
8  col.insert(make_pair("Bob", u2));
9
10 col["Ana"]; // Devuelve u1
11 col["Laura"]; // Devuelve NULL;
12
13 col.erase("Ana"); // Borro u1 de col
14 col.erase("Bob"); // Borro u2 de col
15 col.empty(); // Devuelve True
```

Observar que en map se hace uso de pares, donde el primer elemento del par es la clave que identifica el par y el segundo es el elemento al que se apunta que es identificado por la clave. Para acceder a ambos miembros utilizamos las siguientes operaciones:

- **first()**: Devuelve la clave del par.
- **second()**: Devuelve una referencia al elemento apuntado en el par.

Ejemplo de uso:

```
1  for (map<string, Usuario *>::iterator it = col.begin(); it != col.end(); ++it) {
2      cout << "Nombre: " << it->first << endl;
3      cout << "Edad: " << it->second.getEdad() << endl;
4  }
```

4.7. Patrones de Diseño – Implementación

4.7.1. Singleton

```
1 // Singleton.h
2 class Singleton {
3 private:
4     static Singleton * instancia;
5     Singleton()
6 public:
7     static Singleton * getInstancia();
8     // otras operaciones //
9 }
```

```
1 // Singleton.cpp
2 #include "Singleton.h" // Incluir la ruta correctamente
3
4 Singleton * Singleton::instancia = NULL;
5
6 Singleton::Singleton() {}
7
8 Singleton * Singleton::getInstancia() {
9     if (this->instancia == NULL)
10         instancia = new Singleton();
11     return instancia;
12 }
13
14 // otras implementaciones //
```

```
1 // Ejemplo de uso
2 #include "Singleton.h" // Incluir la ruta correctamente
3
4 int main() {
5     Singleton * ms = Singleton::getInstancia();
6     ms->operacion();
7     return 0;
8 }
```

4.7.2. Factory

Utilizaremos Factory principalmente para permitir el acceso a los controladores del sistema desde la capa de presentación sin que esta quede acoplada a los controladores mediante el uso de interfaces de sistema.

Para ello, presentamos tres alternativas:

1. Que los controladores sean Singleton y simplemente sean accedidos mediante **getInstance()** desde la Fabrica.

```
1  ISistema1 Fabrica::getISistema1() {  
2      return Controlador1.getInstance();  
3  }
```

2. Que los controladores no sean Singleton y que la Fabrica sea la que mantiene una única referencia a cada uno de ellos, controlando su unicidad.

```
1  ISistema1 Fabrica::getISistema1() {  
2      if (this->sistema1 == NULL)  
3          this->sistema1 = new Controlador1();  
4      return this->sistema1;  
5  }
```

3. Que los controladores no sean Singleton y que la Fabrica no mantenga referencias, sino que devuelva una nueva instancia del controlador cada vez.

```
1  ISistema1 Fabrica::getISistema1() {  
2      return new Controlador1();  
3  }
```

Ejercicio:

Consideremos la siguiente interfaz junto con el controlador que la implementa, implementemos Factory y Singleton y veamos como puedo utilizar una interfaz desde la capa de presentación.

```
1 // ISistema.h
2 class ISistema {
3 public:
4     virtual void login() = 0;
5     virtual void logout() = 0;
6     virtual void newUsr() = 0;
7     virtual void opUsr() = 0;
8 }
```

```
1 // ControladorSistema.h
2 #include "ISistema.h"
3
4 class ControladorSistema: public ISistema {
5 private:
6     static ControladorSistema *instancia;
7     ControladorSistema();
8 public:
9     static ControladorSistema *getInstancia();
10
11     // Implementa ISistema
12     void login();
13     void logout();
14     void newUsr();
15     void opUsr();
16
17     // Otras operaciones //
18 }
```

```
1 // Fabrica.h
2 #include "ControladorSistema.h"
3 #include "ISistema.h"
4
5 class Fabrica {
6 private:
7     static Fabrica * instancia;
8     Fabrica();
9 public:
10     static Fabrica * getInstancia();
11     ISistema * getISistema();
12 };
```



```

1 // ControladorSistema.cpp
2 #include "ControladorSistema.h"
3
4 ControladorSistema * ControladorSistema::instancia = NULL;
5
6 ControladorSistema::ControladorSistema() {}
7
8 ControladorSistema * ControladorSistema::getInstancia() {
9     if (this->instancia == NULL)
10         instancia = new ControladorSistema();
11     return instancia;
12 }
13
14 void ControladorSistema::login() {
15     // ... //
16 }
17
18 void ControladorSistema::logout() {
19     // ... //
20 }
21
22 void ControladorSistema::newUsr() {
23     // ... //
24 }
25
26 void ControladorSistema::opUsr() {
27     // ... //
28 }

```

```

1 // Fabrica.cpp
2 #include "Fabrica.h"
3
4 Fabrica * Fabrica::instancia = NULL;
5
6 Fabrica::Fabrica() {}
7
8 Fabrica * Fabrica::getInstancia()
9 {
10     if(instancia == NULL) {
11         instancia = new Fabrica();
12     }
13     return instancia;
14 }
15
16 ICompras * Fabrica::getICompras()
17 {
18     return ControladorSistema::getInstancia();
19 }

```

```

1 // main.cpp
2 #include "Fabrica.h"
3
4 int main() {
5     Fabrica *fab = Fabrica::getInstancia();
6     IUuarios *opsSistema = fab->getIUuarios();
7
8     opsSistema->newUsr();
9     opsSistema->opUsr();
10    opsSistema->logout();
11
12    return 0;
13 }

```

4.7.3. State

```
class Contexto; // Forward declaration

class Estado {
protected:
    Contexto * contexto_;

public:
    virtual ~Estado() {}

    void setContexto(Contexto *contexto) {
        this->contexto_ = contexto;
    }

    virtual void evento1() = 0;
    virtual void evento2() = 0;
};

class Contexto {
private:
    Estado *estado_;

public:
    Contexto::Contexto(Estado * estadoInicial) {
        this->estado_ = nullptr;
        this->cambiarEstado(estadoInicial);
    }

    ~Contexto() {
        delete state_;
    }

    void Contexto::cambiarEstado(Estado * nuevoEstado) {
        if (this->estado_ != nullptr)
            delete this->estado_;
        this->estado_ = nuevoEstado;
        this->estado_->setContexto(this);
    }

    void evento1() {
        this->state_->evento1();
    }

    void evento2() {
        this->state_->evento2();
    }
};
```

```

class EstadoConcreto1 : public Estado {
public:
    void evento1() override {
        // EstadoConcreto1 procesa evento1
        this->contexto->cambiarEstado(new EstadoConcreto2);
    }

    void evento2() override {
        // EstadoConcreto1 procesa evento2
    }
};

class EstadoConcreto2 : public Estado {
public:
    void evento1() override {
        // EstadoConcreto2 procesa evento1
    }

    void evento2() override {
        // EstadoConcreto2 procesa evento2
        this->contexto->cambiarEstado(new EstadoConcreto2);
    }
};

int main() {
    Contexto * contexto = new Contexto(new EstadoConcreto1());

    contexto->evento1();
    contexto->evento2();

    delete contexto;
    return 0;
}

```

4.7.4. Strategy

```
class Strategy {
public:
    virtual ~Strategy() = default;
    virtual tipoRetorno ejecutar(datos) = 0;
};

class Contexto {
private:
    unique_ptr<Strategy> strategy;

public:
    Contexto(Strategy * estrategiaDefault) {
        this->setEstrategia(estrategiaDefault);
    }

    void setEstrategia(Strategy * nuevaEstrategia) {
        this->strategy = nuevaEstrategia;
    }

    void Contexto::operacion() {
        if (this->strategy)
            strategy->ejecutar(datos) // No se como lo va a solucionar.
        else
            error // No se seteo la estrategia
    }
};

class EstrategiaConcreta1: public Strategy {
public:
    tipoRetorno ejecutar(datos) override {
        // implementación //
        return resultado;
    }
};

class EstrategiaConcreta2: public Strategy {
public:
    tipoRetorno ejecutar(datos) override {
        // otra implementación //
        return resultado;
    }
};

int main() {
    Contexto * contexto = new Contexto(new EstrategiaConcreta1());

    contexto->evento1();
    contexto->evento2();

    contexto->setEstrategia(new EstrategiaConcreta2());
    contexto->evento1();

    delete contexto;
    return 0;
}
```

4.7.5. Composite

```
class Componente {
protected:
    Componente *padre_;

public:
    virtual ~Componente() {}

    void setPadre(Componente *padre) {
        this->padre_ = padre;
    }

    Componente *getPadre() const {
        return this->padre_;
    }

    virtual void Agregar(Componente *componente) {}
    virtual void Remover(Componente *componente) {}

    virtual bool esCompuesto() const {
        return false;
    } // Comportamiento por defecto, se sobrescribe en Compuesto

    virtual void operacion() = 0;
};

class Hoja: public Componente {
public:
    void operacion() override {
        // implementacion de operacion //
    }
};

class Compuesto: public Componente {
protected:
    list<Componente *> hijos_;

public:
    void Agregar(Componente *componente) override {
        this->hijos_.push_back(componente);
        componente->setPadre(this);
    }

    void Remover(Componente *componente) override {
        hijos_.remove(componente);
        componente->SetParent(nullptr);
    }

    bool esCompuesto() const override {
        return true;
    }

    void operacion() override {
        for (const Component *c : children_)
            c->Operacion();
    }
};
```

```
int main() {
    Componente *hojaSimple = new Hoja;
    hojaSimple->operacion(); // Tengo una hoja simple

    Componente *arbol = new Compuesto;

    Componente *rama1 = new Compuesto;
    Componente *hoja1 = new Hoja;
    Componente *hoja2 = new Hoja;
    rama1->Agregar(hoja1);
    rama1->Agregar(hoja2);

    Componente *rama2 = new Compuesto;
    Componente *hoja3 = new Hoja;
    rama2->Agregar(hoja3);

    arbol->Agregar(rama1);
    arbol->Agregar(rama2);

    arbol->operacion(); // Tengo un arbol compuesto

    delete hojaSimple;
    delete arbol;
    delete rama1;
    delete rama2;
    delete hoja1;
    delete hoja2;
    delete hoja3;

    return 0;
}
```

4.7.6. Template Method

```
class ClaseAbstracta {
public:
    void metodoPlantilla() const {
        this->OperacionBase1();
        this->OperacionesRequeridas1();
        this->OperacionBase2();
        this->Gancho1();
        this->OperacionesRequeridas2();
        this->OperacionBase3();
        this->Gancho2();
    }

protected:
    // Grueso del metodo, ya tienen implementacion:
    void OperacionBase1() const {
        // implementacion //
    }

    void OperacionBase2() const {
        // implementacion //
    }

    void OperacionBase3() const {
        // implementacion //
    }

    // Operaciones de las subclases, tienen que ser implementadas:
    virtual void OperacionesRequeridas1() const = 0;
    virtual void OperacionesRequeridas2() const = 0;

    // Operaciones "gancho", tienen implementacion por defecto pero pueden ser
    // sobreescritas:
    virtual void Gancho1() const {}
    virtual void Gancho2() const {}
};
```

```

// Usualmente las clases concretas sobreescriben algunas de las operaciones de la clase
base

class ClaseConcreta1: public ClaseAbstracta {
protected:
    void OperacionesRequeridas1() const override {
        // implementacion //
    }

    void OperacionesRequeridas2() const override {
        // implementacion //
    }
};

class ClaseConcreta2: public ClaseAbstracta {
protected:
    void OperacionesRequeridas1() const override {
        // implementacion //
    }

    void RequiredOperation2() const override {
        // implementacion //
    }

    void Gancho1() const override {
        // implementacion //
    }
};

int main() {
    ClaseConcreta1 *clase = new ClaseConcreta1;
    clase->metodoPlantilla();

    ClaseConcreta2 *otraClase = new ClaseConcreta2;
    otraClase->metodoPlantilla();

    delete clase;
    delete otraClase;
    return 0;
}

```


4.7.7. Proxy

```
class interfazSujeto {
public:
    virtual void operacion() const = 0;
};

class Sujeto: public interfazSujeto {
public:
    void opreacion() const override {
        // implementacion //
    }
};

class Proxy: public interfazSujeto {
private:
    Sujeto *sujeto_real_;

    // Se verifica el acceso antes de realizar una solicitud real
    bool verificarAcceso() const {
        // implementacion //
    }

public:
    Proxy(Sujeto *sujeto_real_){
        this->sujeto_real_ = sujeto_real_;
    }

    ~Proxy() {
        delete sujeto_real_;
    }

    void operacion() const override {
        if (this->verificarAcceso())
            this->sujeto_real_->operacion();
    }
};

int main() {
    Sujeto* sujeto_real = new Sujeto();
    Proxy* proxy = new Proxy(sujeto_real);

    proxy->operacion();

    delete proxy;

    return 0;
}
```

4.7.8. Observer

```
class IObserver {
public:
    virtual ~IObserver(){};
    virtual void Notificar(datos) = 0;
};

class Subject {
private:
    list<IObserver *> suscriptores;
public:
    void Agregar(IObserver *observer) {
        suscriptores.push_back(observer);
    }

    void Eliminar(IObserver *observer) {
        suscriptores.remove(observer);
    }

    void Notificar() {
        list<IObserver *>::iterator it = this->suscriptores.begin();
        while (it != this->suscriptores.end()) {
            (*it)->Notificar(datos);
            ++it;
        }
    }

    void operacionQueNotifica() {
        // implementacion //
        this->Notificar(datos);
    }
};

class ObservadorConcreto : public IObserver{
private:
    // ... //
public:
    // ... //

    void Notificar(datos) {
        // Mostrar/Manipular datos //
    }
};

// Opcionalmente se puede hacer que el constructor de ObservadorConcreto se agregue como
// observador a un objeto automaticamente.

int main() {
    Subject *subject = new Subject;
    ObservadorConcreto *observer1 = new ObservadorConcreto(*subject);
    ObservadorConcreto *observer2 = new ObservadorConcreto(*subject);
    ObservadorConcreto *observer3 = new ObservadorConcreto(*subject);

    subject->Agregar(observer1)
    subject->Agregar(observer2)
    subject->Agregar(observer3)

    subject->operacionQueNotifica();

    subject->Eliminar(observer2);

    delete subject;
    delete observer1;
    delete observer2;
    delete observer3;

    return 0;
}
```

4.7.9. Adapter

```
class InterfazCliente {
public:
    virtual ~Target() = default;
    virtual tipoRetorno operacion(data) const {
        // Implementacion por defecto //
        return resultado;
    }
};

class Servicio {
public:
    tipoRetorno operacionServicio() const {
        // Implementacio //
        return resultado;
    }
};

class Adaptador : public InterfazCliente {
private:
    Servicio *adaptado;

public:
    Adapter(Servicio *adaptado) {
        this->adaptado = adaptado;
    }

    tipoRetorno operacion(data) const override {
        // Implementacion //
        // Toma cosas de this->adaptado y las convierte para que sean compatibles //
    }
};

int main() {
    Servicio *aAdapter = new Servicio;
    aAdapter->operacionServicio(); // Pasan cosas raras o no funciona

    Adaptador *adaptador = new Adaptador(aAdapter);
    adaptador->operacion();

    delete aAdapter;
    delete adaptador;

    return 0;
}
```